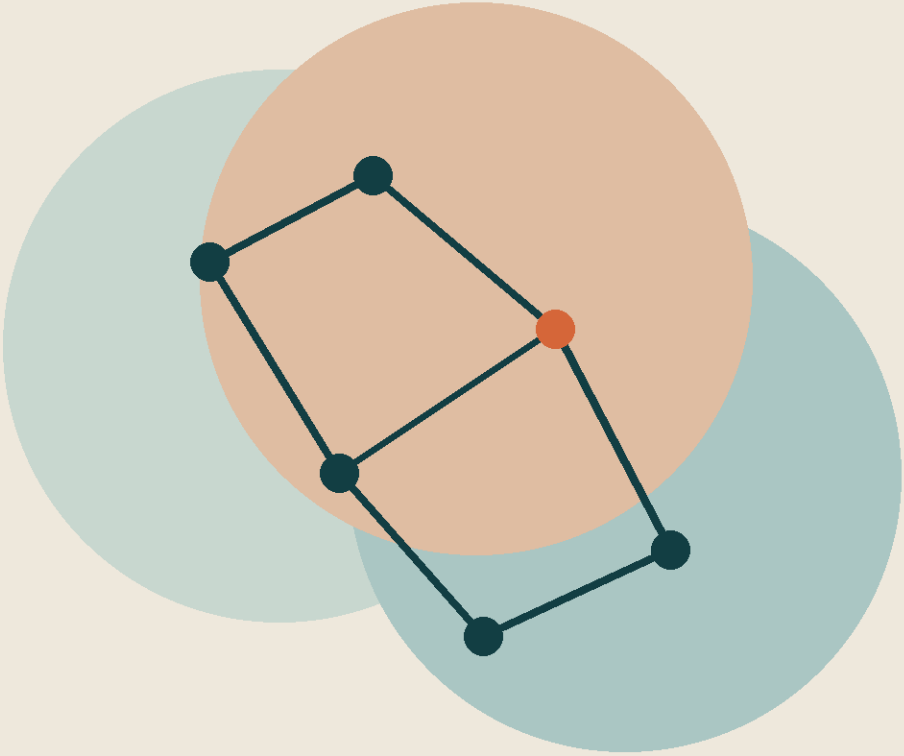
SWM

SYMBOLIC WORLD MODELS

How an AI system preserves the meaning of a situation



PATTERNS · INSTANCES · REASONING

FOURTH EDITION

Symbolic World Models

How an AI system preserves the meaning of a situation

Patterns, instances and reasoning in neurosymbolic systems

Fourth edition
11 November 2026

Copyright

© 2026 Axiologic Research SRL. All rights reserved.

Axiologic Research SRL created and contributed the initial version of this book as part of the SCRIPTA use case in the Achilles project.

Edition 2 · published 2026-11-11 on ScriptaHub (scriptahub.com).

Foreword

An assistant can help us plan a trip, understand a document or organize a project. Almost always, things become interesting after the first answer. We change a date, withdraw an assumption, return to yesterday's discussion or discover that two people use the same word with different meanings. At that point, we need the system to retain the situation we are discussing and understand exactly what has changed within it.

This book explores an architecture for that continuity. We call it a symbolic world model, or SWM, after the English expression Symbolic World Model. The idea is to connect language to explicit representations of situations, so that a system can track relationships, calculate consequences, compare possibilities and formulate relevant questions. The neural part can propose what a message means. The symbolic part organizes the interpretation and tracks its consequences. Interpreters and execution code turn descriptions into calculations and, where appropriate, into actions on real systems.

The book's thread passes through different worlds. A trip helps us see dependencies between events. An organization shows why customer relations, production and research require distinct representations. A story makes the difference between what happened and what a character knows visible. A program lets us investigate an action before executing it. The architecture presented at the beginning is then broken down into mechanisms: threads and circuits, matching, composition, validation, updating and execution. A building connected to sensors shows us how predictions return to

reality to be compared with observations. These situations are constructed for explanatory purposes; the technologies and research discussed are identified in the bibliography.

Two concepts will hold this diversity together: **the world pattern**, which describes what we can reuse across situations, and **the world instance**, which retains a particular case. We will see how the same reality can participate in several worlds and how their connections can be represented without making their meanings uniform. The stakes are practical: being able to ask a system why it supports a conclusion, what would change it and what we should find out or do next.

Contents

Foreword

1 When a conversation needs a world

2 Several worlds within the same reality

3 How language takes a precise form

4 Reasoning opens possibilities

5 A world that can change its mind

6 The right question at the right moment

7 Actions we can investigate before executing them

8 Several representations of the same world

9 When the representation tracks the real world

10 How worlds work together

11 How we learn better patterns

12 Worlds where thinking this way is worthwhile

Sources and recommended reading

1 When a conversation needs a world

“We’re moving departure to Friday.” The Romanian sentence has four words and seems to ask for a simple change. Behind it, however, lies a trip already discussed: a train, a booked room, a meeting and someone who was going to meet us at our destination. The person receiving the message understands that some of these arrangements need reconsidering. They may respond immediately, but may also notice something we overlooked: the meeting is no longer possible on Friday, even though the train and hotel are available.

An AI assistant encounters the same problem. Recognizing the day and the verb “to move” is only the beginning. To help, it must establish which departure we mean, which commitments depend on it and whether we are asking to explore an alternative or change the reservations. A single remark thus opens three connected questions: what the message means, what follows from it and what we are actually authorized to change.

These questions lead us toward symbolic world models. They start from the need to maintain a representation of the situation, which the conversation can extend and transform. The intended gain is continuity of meaning between what the user says, what the system calculates and what happens afterward.

The architecture behind an answer

If we were building this assistant, where would we put the information that the hotel had been chosen for its proximity to the meeting? And which component should react when the meeting moves? These questions turn an intuition into an architecture. The system needs an explicit memory of the situation, a description of the relationships it can interpret and components that perform operations on them. A useful answer emerges through collaboration between these parts.

We will use **world pattern** to mean the reusable description of the objects, relationships, conditions and operations within a family of situations. **The world instance** retains the particular case: the trip being discussed, its options and the information we rely on. The pattern says, for example, how to check a connection between two transport legs. The instance says which legs they are, when they take place and what transfer duration we know. These are the book's two central concepts. Interpreters and executors are the software components that use them.

The representation can be organized through **threads** and **SOP circuits**. A thread identifies an element and describes its properties: a person, a meeting, an observation or a requested operation. References connect threads into a circuit. In our trip, a reservation points to the appropriate person and transport leg, while the attendance check points to the arrival and the start of the meeting. The pattern establishes the meaning of these connections. The cir-

cuit provides a structure that a program can traverse, query and transform; its mere existence does not, however, make any reservation.

Viewed as a whole, the architecture has one main path and two important return paths. A message produces a proposed interpretation, which is checked in the context of an instance, and the adopted change triggers the relevant calculations. If a decisive distinction is missing, the path returns to the dialogue. If an external action is requested, the path passes through an executor and returns through the observed result. The figure shows these relationships; the same application can perform several roles without confusing their responsibilities.

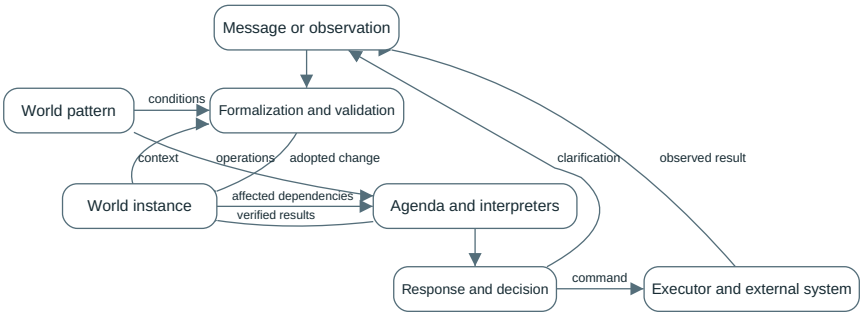


Figure 1. The instance retains the situation and the results. Clarification and external effects return as inputs for a new processing cycle.

Let us follow the mechanism using the message “we’re moving departure to Friday”. At the input, we retain the message and the context in which it was received. A context selector looks for plausible instances based on the active discussion, the people mentioned and the events already connected to them. A neural model proposes that the message concerns the trip under discussion, identifies the date and interprets the intention. The result of this stage is a **proposed change**: the target object, the requested change, the source and the assumptions used. It is not yet the new state of all the reservations.

This proposal is evaluated on a working copy or a list of differences from the current version of the instance. The validator checks whether the object exists, whether the new value makes sense for the property being changed and whether the references lead to the expected elements. A meeting's start requires a point in time; a participant requires a reference to a person; a duration must be interpreted in the correct unit. If "Friday" could refer to two relevant dates, the system retains the alternatives and asks for the clarification that changes the result. Validation does not silently fill in what the message has not established.

After the structural checks, the interpreters evaluate the relevant conditions. In our case, changing the departure changes the arrival, and the arrival affects attendance. The engine does not have to repeat every calculation about every trip. It consults the dependency links and places the affected operations on a **work agenda** . The agenda is the list of calculations still to be performed, each associated with the data and version for which it is requested. A temporal interpreter recalculates the arrival time; another calculation checks the margin before the meeting. If the result changes a conclusion used further on, that conclusion in turn places other operations on the agenda.

Here we must distinguish two kinds of connection. The fact that the meeting and the trip refer to the same person is a semantic relationship. The fact that assessing attendance requires the arrival time is a computational dependency. Only the second establishes that one result must be available before another calculation. From the circuit describing the situation, we construct or select the dependencies required by the current operation. In the following chapters, we will see how some calculations can run in parallel, how others require an order and how we handle rules that reactivate one another.

Results must retain their origins. The assessment "we won't arrive in time" is connected to the new arrival, the transfer duration and the meeting time. This justification enables two operations: explaining the answer and revising it when one of the premises

changes. In memory, we can retain the threads in a collection addressable by identifier, an index by type and properties, plus reverse links showing where each piece of information is used. These are ways of organizing the same data. They enable rapid retrieval and local updating without adding a new notion of world.

At the end of this path, application control chooses what kind of result to provide. If the question concerned feasibility alone, we present the conclusion and the decisive premises. If there are several options, we retain the scenarios. If the information separating them is missing, we formulate a clarification. If the user actually requested the change, we prepare the operation that the reservation service can accept. The choice is not inferred solely from the fact that a scenario is possible: it also depends on the intention, preferences and authority expressed in the interaction.

Adopting a change in the instance and producing an external effect are two operations with different results. In memory, we can immediately record the decision “we want to depart on Friday” and the fact that the ticket change is pending. The executor sends the request to the service, with the operation’s identity and the necessary conditions. If the service confirms, the result enters as a new event and updates the reservation’s state. If it refuses, the instance retains the existing reservation and the reason for refusal. If the response is lost, the result remains unknown until a check is made; the system does not invent confirmation from its own intention.

Persistent memory makes it possible to continue this process. After closing the application, we must be able to recover the instance, the pattern version, the important justifications and the external operations that remain unresolved. The current state can be retained together with a log of relevant changes. A calculation begun on an old version is not automatically adopted after the user has changed the date again: the result must be checked against the current premises. Thus, consistency is not merely a check at loading time, but a property monitored throughout updates.

Through this architecture, the SWM becomes the part that retains the semantic organization of the activity. The neural model proposes meanings and recognizes objects. Interpreters calculate consequences for the descriptions they understand. Application control chooses the next step, and executors interact with services or devices. We can begin with a single pattern, an instance store and a few well-defined operations. Expansion adds other patterns and interpreters through the same connection points: identified data, explicit conditions and results that retain their meaning.

The situation continues after the sentence has ended

The Thursday departure had been chosen to arrive the evening before Friday's meeting. The architecture is already tracking the effects of a timetable change; now a deeper change appears, in the reason for the trip.

A conversation can also change how we view the goal. The user adds: "The meeting can take place online; what matters is being there for the weekend." The Thursday departure had been justified by a condition that has just lost its importance. A system that retains justifications can reopen the choice of trip without treating the entire plan as wrong. The hotel may remain suitable, transport from the station may need a different time, and the meeting may take a different organizational form.

An important observation for designing an assistant's memory follows from this. A useful memory contains both what we chose and the reason the choice made sense. If the reason changes, memory can indicate what is worth reconsidering. This property is just as relevant in document analysis or a research project: a conclusion that survives after its premise has been withdrawn can become more dangerous than forgotten information.

At this scale, symbolic representation resembles good organization of intellectual work. We have identifiable objects, statements about them and relationships we can inspect. Yet the aim is for programs to be able to use these relationships. The system can track dependencies, check a temporal condition or construct a variant of the plan. Explanation and calculation use the same organization of the situation, allowing the result to be connected to the steps that produced it.

The same life contains different worlds

A person's phone may contain a calendar, a novel in progress, family messages and household consumption data. All concern that person, but each requires different distinctions. In the calendar, overlapping intervals and participants' agreement matter. In the novel, continuity of events and what each character knows matter. In the home, sensor observations, changes in temperature and commands sent to equipment matter. Belonging to the same person does not, by itself, provide a common way to interpret all this information.

We will use the word "world" for a boundary within which certain objects, relationships and changes suffice for a family of questions. The world of a trip lets us discuss routes, arrivals and connections. The world of a novel lets us discuss characters, events and perspectives. The boundary is chosen according to what we want to understand or do; it can become richer when we encounter a question requiring a new distinction.

An example from reading makes this idea easy to grasp. The reader learns that a letter has been hidden, but the main character does not yet know. If the story makes the character act on the letter's contents before learning them, a break in continuity appears. To identify it, we must represent the event and the character's access to information separately. A list of the novel's facts could be correct

yet insufficient for this question. The appropriate representation also retains the perspective from which each fact can influence an action.

In a home, we encounter a different difficulty. “The room must be warm on arrival” expresses an intention, the measured temperature expresses an observation, and the probable arrival time expresses an estimate. The system must relate them to decide when to turn on the heating. If the train is late, the time available for heating changes. If a window remains open, the physical conditions of the prediction change. The two changes reach the same decision through different relationships.

The examples show us why a plurality of worlds is useful. We are not trying to cram everything into a single vocabulary in which “available”, “known” and “confirmed” have only one meaning. We build representations suited to the activity and specify the connections between them. The arrival time can be shared by the worlds of the trip and the home without a rule about tickets becoming a rule about temperature.

This choice also has a practical criterion. Distinguishing worlds is worthwhile when the same information changes its operational meaning, when answers require different evidence or when changes follow different rules. Two closely related questions in the same calendar can use the same world. Investigating a hypothesis and confirming an order may need distinct worlds, even if they concern the same product. The system’s generality will come from how it can organize and connect these differences.

What we reuse and the case we experience

When planning a new trip, we do not rediscover each time that we must arrive before the next train departs. We use previously learned relationships and apply them to new events. When reading a new novel, we assume that a character retains their identity between two scenes and that information must reach them along

some path. This reusable background can be modified by context: a story may allow time travel, and a route may include a transfer organized under different rules. Each case contains something general and something particular.

The distinction introduced in the architecture now takes on a broader meaning. The world pattern describes what we can reuse to interpret situations and changes. The world instance retains the particular case: the trip currently being discussed, the novel we are analyzing or the home's situation this evening. The instance can contain certain information, alternatives still open, assumptions and a history. The pattern allows this information to be related and used in reasoning.

The distinction appears in what we can change. If the train arrives at a different time, we update the trip instance. If the pattern assumed that every change of train takes place at the same station, discovering a transfer across the city requires enriching it: it must also be able to represent travel between stations. The duration of the actual journey remains information about those places and conditions, even if we use it in several trips. Reusing information does not automatically turn it into a rule of the world.

The pattern actively participates in understanding the instance. It can show that the transfer duration is missing, that a late arrival calls a meeting into question or that a character could not yet have known something. It can guide a question and make a consequence not literally present in the message visible. This is the interest of the SWM architecture: reusable knowledge becomes an instrument for interpreting and exploring a case, while the case can provide reasons to improve reusable knowledge.

In a neurosymbolic system, the neural model can propose that "we're moving departure" concerns a particular trip and a particular date. The instance provides the context in which we check that proposal. The pattern enables consequences to be calculated, and a reservation service can execute the change if it is requested and permitted. Each component contributes to the same thread of

meaning. The reader should be able to follow how the system moved from the sentence to the choice and then to the observed result.

An organization makes this problem even more interesting. There, the same conversation can move within seconds from a person's availability to a product's readiness and the value of experimental evidence. To understand such a dialogue, we must first see how several worlds can exist within the same reality.

2 Several worlds within the same reality

A customer asks whether they can try an electric bicycle featured in a company message next week. The calendar has an available slot. The team preparing the bicycles has one unit almost finished. The researchers have obtained a promising result for a new battery. At first glance, the information seems complementary. If the assistant compresses it into the statement "the product is ready", the decisive information may disappear: the unit prepared for the customer uses the existing battery, while the new result concerns a different prototype.

None of the teams necessarily said anything false. They were answering different questions. The person scheduling the meeting was asking when the customer and adviser could attend. Production was tracking whether the specific unit could be made available. Research was trying to discover under what conditions a par-

ticular technical variant works. The assistant must retain these differences to construct a shared answer that promises no more than the information supports.

A world's boundary follows the meaning of the question

In the world of scheduling, a reservation can occupy a time slot, have participants and be confirmed, moved or cancelled. The question "can we do Friday?" concerns calendar compatibility and the meeting's conditions. This world's pattern describes such relationships. The instance can contain the team's schedule for a period, with several visits and their shared resources; this customer's visit is an object within that situation. We can thus detect that two visits require the same person simultaneously. The fact that the bicycle is manufactured elsewhere becomes relevant when its availability is a condition of the visit.

In the world of product preparation, the same visit appears as a deadline for a requirement: a particular unit must be in a particular state. Here, the bicycle's physical identity, its configuration, the operations performed and the checks required by company procedure matter. "Ready" means those conditions are satisfied for that unit. A change of adviser may have no influence at all on this world. A change to the configuration requested by the customer may change its entire plan.

The research world organizes information differently. It connects a hypothesis to experiments, test conditions and observations. A favorable result may support the claim that a variant is worth investigating further. For it to become relevant to the customer's visit, we must know whether it concerns the configuration to be used and what conclusion it permits. A figure for a prototype's range, obtained on a particular route, remains connected to those conditions even if the same product family shares a commercial name.

The three worlds do not necessarily correspond to three departments. A single person can work in all of them. A team can maintain several patterns, and an application can host them together. The criterion for separation is the meaning of the relationships and operations. “Confirmed” in a calendar means agreement on a meeting; “confirmed” in research requires specifying the claim and the evidence; “confirmed” in an execution system may mean that a device has reported carrying out a command. A shared field with the same name does not resolve these differences.

This observation has an important precedent in software design. The Domain-Driven Design tradition explicitly addresses situations in which different models use similar vocabulary and requires the relationships between contexts to be described. In an analysis of integrating an AI component, Eric Evans discusses precisely the separation of the language model’s contribution from the application’s models and responsibilities. [S28] The parallel is useful for SWM: the neural part can help recognize a situation, while a decision’s meaning remains connected to the context in which it is made.

A good boundary saves unnecessary explanations and makes important connections visible. We do not represent battery chemistry in every appointment. Instead, we retain the connection between the visit, the promised configuration and the available unit. If the discussion reaches a performance claim, we can trace the evidence in the research world. The representation remains rich enough for the actual questions without making every detail of the organization mandatory everywhere.

The pattern also encompasses what can happen

Two calendars can have exactly the same fields yet organize different activities. In one, a request immediately occupies the time slot. In the other, the slot is occupied only after acceptance. If we repres-

ent only the fields “start”, “end” and “participant”, this difference disappears. It reappears when two people request the same time and the system must decide what happens.

For this reason, the world pattern includes behavior, not just the shape of information. It says what consequences accepting a request has, what remains unchanged and what conditions must be checked. For the customer’s visit, it can describe the transition from intention to proposal and then to a confirmed appointment. A concrete instance may still be at the first stage. Its presence in the system is useful even if not all details are known: we can investigate alternatives and identify precisely the information on which the next step depends.

The pattern can contain rules with different grounds. The relationship between a meeting’s duration and its times is a definition. A procedure requiring an adviser’s presence is an organizational rule. An estimate of battery range describes a phenomenon and is compared with observations. These rules may participate in the same decision, but they do not change for the same reasons. An ambiguous definition requires clarification of terms; a procedure requires a decision by the responsible person; a prediction that repeatedly deviates from measurements may require different parameters or a different model.

The distinction matters when reality does not follow the plan. An adviser may mistakenly confirm two simultaneous visits. The system must be able to represent and flag this situation. The rule that should have prevented double confirmation remains useful, but does not justify deleting one of the reports to obtain an apparently impeccable calendar. The world must also encompass the situations we want to understand, repair or prevent.

An instance can change over time without losing its identity. The team’s tracked schedule continues to exist when we move a meeting, and within it the visit can retain its own identity. We can thus ask “why did we move it?” and “who was informed?”. If we only want to compare two times, we can construct two schedule scenari-

os starting from the same instance. Each retains its own assumption, and choosing one does not retroactively change the other's history.

In this sense, the instance is richer than a snapshot of the current state. It can represent what we know about the case, what we have assumed and what still awaits confirmation. Its coherence means that the statements used together have an interpretation compatible with the pattern. Their truth about reality rests on observations and evidence. The two meet in the system's activity: rules discover mismatches, while new information allows an interpretation to be selected or revised.

The connection between worlds is itself something to explain

Imagine that the assistant learns "the battery test succeeded" and automatically changes the customer's visit to "confirmed". The error occurs in the passage between worlds. The experimental result may be relevant to preparing a demonstration, but does not by itself establish the unit's availability or the customer's agreement. The correct connection must say what statement is transferred, about which object and for which decision.

We can construct this connection in intelligible steps. The visit refers to a promised configuration. A unit is prepared in that configuration. A particular check concerns that unit or a class of units for which transferring the result is justified. Only then can information about readiness satisfy a condition of the visit. When such connections are explicit, the assistant can also identify what is missing: perhaps we have a good result but do not know whether the unit intended for the customer uses the same version.

Identity is crucial in this passage. The name of a bicycle family, the version of a design and the number of a physical unit designate different things. A test performed on a prototype must not automatically be associated with every product bearing the same name. In

other domains, the problem reappears in other forms: a document's version versus its title, a person versus the role they occupy, a product's recipe versus the actual batch. The system must retain what is shared and what remains particular.

Sometimes we compose patterns into a more encompassing one. To prepare a visit, we can connect calendar rules to product availability conditions. At other times, we keep the worlds distinct and exchange only the necessary information. Research can continue at its own pace, while production can receive a conclusion accompanied by its conditions. The choice depends on how often they must be considered together and on responsibility for the information. Both forms of organization are compatible with the book's two concepts.

The same principle appears in a family planning a holiday. The work calendar, school schedule and trip do not become a single domain simply because they influence the same dates. Approval of leave and availability of a flight are different conditions. A shared solution must respect both, and a change to one must reach the choices that depend on it. A useful assistant retains these connections without assuming that it has authority over every calendar.

How we compose patterns without mixing meanings

A family wants to leave on holiday after classes end and arrive before reception closes. The school calendar, transport and accommodation can be correctly described separately, yet the chosen combination may still be impossible. A useful composition must make it possible to discover this incompatibility. Bringing all the information together in one file provides access to data; connecting the conditions makes joint reasoning possible.

The first step is to specify what contribution we request from each pattern. From the calendar, we use the time from which the people can leave. From transport, we use the places, departures, arrivals

and connection conditions. From accommodation, we use the location and the interval during which arrival can be accepted. These contributions form the interface through which we compose the worlds: the declared part of each description on which the others can depend. Details about assessing pupils or managing rooms are unnecessary for the current question.

Next comes identifying the roles that must correspond. The people leaving must be those to whom the calendar availability applies. The relevant arrival is at the accommodation, after the transfer, not simply landing in the city. Intervals must be related to compatible dates and time zones. If one source measures the transfer in minutes and another in seconds, the connection must include the conversion. These adaptations are explicit operations of the composition; similar field names do not replace them.

Once the connections are established, we combine the conditions for the chosen scenario. Departure cannot precede the people’s availability. The transfer must begin after the preceding leg arrives. Arrival at the accommodation must fall within the accepted interval, or another applicable procedure must exist. The interpreter looks for values that satisfy all these conditions together. If it finds a combination, it produces a scenario with the connections and premises used. If it does not, it must distinguish an incompatibility demonstrated within the explored domain from a search stopped before resolution.

The composite pattern thus includes more than the union of the initial descriptions. It adds the relationships through which one world’s result becomes a condition in another. “We arrive in time to check in” belongs in isolation neither to the transport timetable nor to reception rules. It is a conclusion constructed where they meet. It must have a justification traversing the same connections. If the flight time changes, we can find the affected conclusion; if information about the neighboring room changes, we have no reason to recompute the route.

Composing behavior requires an additional clarification. Hotel confirmation and transport confirmation are distinct operations managed by different systems. We can require a plan to contain both confirmations before it is presented as settled, but this condition does not turn them into a single indivisible operation. If the first succeeds and the second fails, the instance must represent the partial result. The plan can provide an alternative or cancellation according to the reservation terms. Synchronization is described and executed through a protocol; it does not appear automatically from connecting two circuits.

When loading the composite pattern, we check whether the interfaces match: the roles exist, the types and units are compatible, and the invoked operations have interpreters. When using it, we check the conditions of the concrete case. A pattern can be well constructed and reject a particular flight-and-hotel combination; this is useful information about the instance. Conversely, a mismatch between two units or two meanings of the same property requires repairing the composition before deciding anything about the trip.

We can use this connection in two ways. When the same problem recurs frequently, we retain a reusable composite pattern and construct instances for other holidays. When the worlds have independent rhythms and responsibilities, we keep the instances distinct and exchange only results through the established interfaces. In both cases, the engineering work is the same at the decisive point: we must know what information passes, what it means to the recipient and which conclusions or operations it can influence.

The plurality of worlds thus becomes a resource for precision. It lets us explain why information is sufficient in one place and still insufficient in another. To turn this idea into a system, we need a form in which objects, references and conditions can be tracked. The next difficulty is reaching that form from the natural, often incomplete way in which we speak.

3 How language takes a precise form

“If we take the eight o’clock train, will we reach the meeting? Keep that option, but don’t change the tickets.” A person recognizes in this message a question about a possible future and a limit on action. We want to explore, then retain the result. The train, meeting and tickets belong to the same conversation, but the sentence does not say the new plan has been adopted. For an assistant, losing this distinction could mean performing an operation that the user has just ruled out.

Formalization begins with such choices of meaning. Only after retaining them can we calculate whether the arrival is compatible with the meeting. The notation we use must make the choices and their connections visible, so that a person can inspect them and programs can use them. SOP Lang offers a compact form for these descriptions. Its interest in this book is that it lets us follow the passage from message to representation, then from representation to interpretation.

A sentence proposes several things at once

The expression “the eight o’clock train” seems a sufficient identifier until two days or two stations appear in the conversation. The neural component can use context to propose the intended train. It can also recognize that “we’ll reach” refers to the trip’s participants and “the meeting” to the event discussed earlier. These associations must be retained as part of the formalization. A correct calculation performed for a different train precisely answers a question the user did not ask.

In a useful representation, we can see the object chosen for each reference and the role of each sentence. The first part of the message introduces a scenario. The second asks for it to be remembered. The last rules out changing the tickets. The trip pattern helps interpretation: we know that a leg has a departure and arrival, that a transfer takes time and that an actual reservation differs from an option. The instance supplies the concrete identities and the information already established in the conversation.

A symbolic check may discover that the proposed train departs from a different city from the trip's origin. It may also show that two trains remain compatible with the message. In the second case, a clarifying response is better grounded than an arbitrary choice. The check's power comes from explicit relationships, while information about language remains necessary to decide whether the representation matches the intention. The two operations can help each other in a repeated process: formalization proposes, checking identifies a mismatch, and context or the user resolves it.

Neurosymbolic research has explored this division concretely. The Neuro-Symbolic Concept Learner constructs a representation of the objects in an image and turns the question into a program operating on that representation. Perception, question interpretation and execution are connected in a learned system. [S10] For our problem, the architectural lesson is that a question can become an inspectable operation on a represented situation. A conversational assistant then adds continuity between messages, changing information and the consequences of actions.

This continuity also requires a trace back to the message. If the question was "could we?", the system must be able to show why the result is a scenario. If the user corrects it with "I meant the next day", the changed reference must reach the dependent calculations. Retaining the language fragment and its interpretation makes the correction locatable. In this way, formalization becomes a proposal that can be examined and improved, rather than a translation that immediately disappears inside the system.

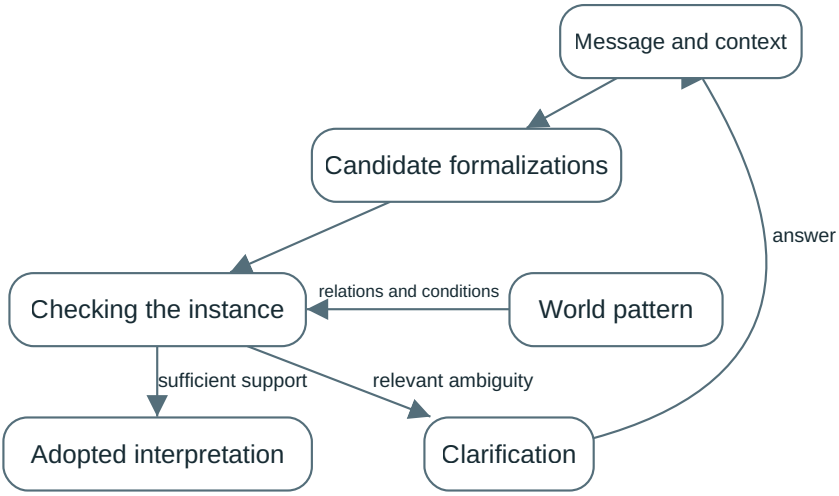


Figure 2. The pattern participates in interpretation, and a clarification can restart checking.

SOP Lang makes connections visible

Take two contacts with the same first name. In a text, both can appear as “Ana”. In an appointment, we must know which one is meant. This is why an identifier differs from a displayed name. It allows the meeting, trip and messages to refer to the same person, even if her name changes or is shared with someone else. This is the first need addressed by the basic form of an SOP declaration. [S1]

@ana person name="Ana"
@tren service label="T1" date="2027-05-14" departure="08:00"
@calatorie trip traveler=$ana service=$tren date="2027-05-14"
@intalnire meeting participant=$ana
date="2027-05-14" start="11:00"

Each declaration is called a thread. @ana gives the first thread an identifier, person specifies its type, and name="Ana" adds a property. The reference $ana connects the trip and meeting to the same

person, while $tren connects that trip to service T1 on the indicated day. A collection of threads forms an SOP circuit. References express connections between threads; their mere presence does not impose an order of calculation or execution.

We can understand the in-memory representation without knowing a library: a registry associates each identifier with a thread, and the thread retains its type and properties. Each property has one or more values. Some are texts or numbers; others are references that can be resolved in the registry. A trip does not contain a copy of the person, but a link to her. Thus, several trips can concern the same person without creating new people. The world instance encompasses this collection and the information needed to interpret it, not just a single thread.

The $ sign matters because a connection and a text have different uses. The reference participant=$ana allows traversal of the relationship to the person. A quoted string such as participant="$ana" retains only those characters. The pattern establishes where a reference is necessary and where text is appropriate. Neither identifiers nor property names determine semantics on their own; they make it possible to express the chosen semantics without the ambiguity of a free-form paragraph. An identifier is local to the space in which we resolve it. Another circuit can use @ana for a different person; joining them requires an explicit decision about identity.

Let us add the scenario data. The following times are chosen for the example, on the same day and in the same time zone. The train arrives at 10:20, and the transfer from the station to the meeting place takes 25 minutes. For the calculation, we use the number of minutes since the start of the day. The instance can also retain the ordinary time format for display; the conversion must respect the same convention across all values used.

@sosire arrival traveler=$ana trip=$calatorie minute=620
@inceput meeting_start meeting=$intalnire minute=660
@deplasare transfer
from=$sosire to=$inceput duration_minutes=25

The from relationship connects the transfer to the relevant arrival, while to connects it to the meeting's start. The 25 minutes are not an unanchored duration. If there is another meeting that day, the links help us avoid accidentally using its transfer. Moreover, a thread spanning several lines is a matter of readability. Temporal order is expressed through properties and relationships, not inferred from position in the file.

SOP can also represent relationships with several participants. A declaration can contain the participant key several times if the meeting vocabulary permits this multiplicity. The pattern must say whether each participant is mandatory, whether everyone's acceptance is necessary and what a missing response means. The syntax provides a common form, while the differences that change behavior remain explicit in interpretation.

This division is useful for extension. A syntax reader can recognize threads, keys and references without knowing a building's physics or a publication's rules. A specialized interpreter then uses the vocabulary of the world concerned. In the same format, we can describe a character and the source of information, a measurement and its sensor, a meeting and its participants. The notation's generality is infrastructure for diversity, while the pattern retains each domain's meaning.

But where do we retain the description of the pattern itself? It too can be expressed through an SOP circuit: threads for type and property definitions, threads describing matching templates, rules, calculations and conditions. We use the same representational form, but in different roles. The thread declaring that a meeting can have participants belongs to the pattern's description; the thread for Ana's meeting belongs to the instance. Declaring a rule about trips does not assert that a particular trip took place.

The interpreter prepares the pattern before using it. It resolves links between declarations and checks that a rule points to existing templates and that a calculation names an available operation. From property definitions, it constructs the record of checks applic-

able to each type. From premises, it constructs indexes showing which rules must be investigated when a fact of a particular type appears. It also prepares the matching steps: which threads are sought, which variables must coincide and which calculations or conditions follow. This is compilation of the pattern into a form convenient for processing; it does not automatically turn every reference into an execution order or invent implementations for the named operations.

A trip rule may require an arrival, the transfer to the intended meeting and its start time. It calculates the time of reaching the meeting and checks whether, together with the required margin, it meets the deadline. The consequence is a favorable temporal assessment for that option, connected to the data used. The rule remains in the pattern; the assessment produced enters the instance. If the duration is missing, the rule does not produce that assessment: absence of the conclusion does not prove lateness. Thus, the same reusable circuit can process many instances without mixing their identities or results.

Matching identifies relationships between threads

Ana may have two meetings on the same day, while someone else may travel on the same train. A search for the words “train” and “meeting” would gather relevant data without telling us which should be used together. Symbolic matching checks precisely this connection. Its inputs are the circuit sought, the available circuit and the rules for interpreting types and values; its result is a collection of complete matches, each with its own bindings between variables and values.

When working with many trips, we need to find the appropriate information without rewriting every identifier. This is where variables appear. In the following search templates, ?cine designates a

value to be found, and using it in two places requires that value to be the same. The ? sign before the declaration's identifier indicates that we have a thread to search for, not an assertion of a new fact.

?persoana_in_drum trip traveler=?cine date=?zi
?eveniment meeting participant=?cine date=?zi start=?ora

The engine begins with threads of the trip type. If it has an index, meaning a registry grouping threads by type or by a property, it need not inspect every object in the world. When it finds Ana's trip, it provisionally notes that cine is the reference $ana and zi is the indicated date. This small association is called a substitution: it says which value can replace each variable in the current match. It also retains the identity of the thread that satisfied the first template, so it can explain where the values came from.

With the substitution thus obtained, the second template seeks a meeting involving the same person on the same day. A meeting involving Dan is rejected even if it starts at the desired time. Ana's meeting on the following day is rejected for the same reason: an already bound variable cannot silently acquire another value. For the compatible meeting, the engine extends the substitution with the time. If two meetings are compatible, two matches result, each retaining the identities of the threads that satisfied the templates. Even if the substituted values coincide, the meetings may be distinct; identities allow each result to be justified.

Suppose the first trip tried has no compatible meeting. The search returns to the choice of trip, abandons that attempt's provisional bindings and tests the next one. This return is called backtracking. It must keep branches separate: the time found for one meeting cannot accidentally remain in another's substitution. The same discipline enables matching larger circuits with several shared conditions. Enumeration is complete when candidates have been exhausted. If we stop at the first result or at the computation limit, the output specifies that other matches may exist.

Variables have a local scope of use: the name cine in another rule must not accidentally become the same unknown. The template represents a condition on the data. Introducing ?ora into it does not change the meeting's actual time. If we obtain no match, we know only that the available data do not satisfy the search. The time may be missing, the day may be wrong or the meeting may not exist. To say which situation we have, we use other checks and information about what we consider complete in the instance.

Precision also requires a convention for values. In SOP, matching literal values preserves their textual form: 5 and 5.0 are not the same match, although a numerical comparison can recognize their equality. Data normalization or explicit calculation must resolve the difference according to the intended meaning. Similarly, two identical names do not prove that the people are identical. Matching applies the declared relationships; it does not replace establishing them.

Semantic connections and calculation order answer different questions

The reference to Ana tells us who attends the meeting. It does not tell us whether we must first check the train timetable or the room's availability. The difference can be lost when we draw every relationship with arrows. To design processing, we must distinguish three structures that can use the same threads without meaning the same thing.

The semantic graph says what the connections represent: Ana travels, the train belongs to the option under discussion, the transfer connects the arrival to the meeting. Here, a graph is simply a set of elements and relationships between them. A computational dependency graph says which values must be known to obtain a result: the temporal check reads the arrival time, transfer duration, meeting time and required margin. The execution plan says in

what order the available operations will be performed and which can take place independently. It is constructed for a question and a state of the data.

The trip data are now visible, but the answer still has to be obtained. For the simple scenario, we add 25 to 620 and obtain 645, or 10:45. We compare this with 660, or 11:00. The resulting margin is 15 minutes. This conclusion concerns the declared times and conditions: the transfer used is the one between the selected train’s arrival and the intended meeting. If we want a minimum margin of 20 minutes, the same calculation leads to a different assessment of the option.

To describe a calculation as well, we can use threads with roles declared by the application’s vocabulary. The example below introduces an instructional vocabulary: time_check names a temporal compatibility check, and its properties indicate exactly the data and condition used. It explains one possible semantics; it does not depend on a library method.

@ajungem_la_timp time_check
arrival=$sosire transfer=$deplasare deadline=$inceput
minimum_margin_minutes=15

The interpreter for this declaration must resolve the references and check that the transfer starts at the indicated arrival and reaches the indicated event. It then reads the values in compatible units and checks the relationship “arrival plus transfer plus minimum margin is no later than the meeting’s start”. For our values, 620 plus 25 plus 15 is 660, so the condition is satisfied exactly at the limit. An explainable result retains the relationship, the values and their origins. When the transfer duration becomes 30 minutes, we can see why the conclusion changes without reinterpreting every thread in the world.

Another interpreter could treat transfer duration as an unknown and infer that it must be at most 25 minutes to respect the chosen margin. The same description of the relationship can thus serve a concrete evaluation or a search for conditions. For this reuse, the

semantics must be specified well enough for interpreters to handle the same values, the same limits and the same meaning of the result.

How do we obtain the calculation plan in concrete terms? We start from the requested result and consult the operation's contract: what it reads, what it produces and under what conditions it applies. We resolve references, identify existing inputs and look for operations that can provide missing inputs. Operations with no unresolved dependencies can be evaluated; their results unlock other operations. If the transfer duration is missing and there is no accepted way to obtain it, the check remains pending with that gap identified. We do not evaluate it as if the duration were zero.

A semantic circuit can have perfectly ordinary circular links: the person points to the organization they belong to, while the organization lists its members. This does not prevent reading the data. A computational cycle, however, requires interpretation: if result A depends on B and B on A, ordering alone is insufficient. We need an explicit resolution rule, a justified iteration or a stop with the dependency unresolved. Simply drawing the circuit does not decide this choice. In the proposed architecture, operation contracts make it possible to separate a semantic relationship from a dependency that blocks calculation.

Composition preserves identities and declares what crosses the boundary

A digital publication is preparing an illustrated article. The editorial circuit describes the text, the chosen images and their positions on the page. Another circuit describes image files, versions and usage approvals. Both may speak of "the cover photograph", but this expression is insufficient to join them: the editorial choice and the approval concern different roles, and the file may have been replaced in the meantime.

The inputs to composition are the two circuits and a declaration of the connections allowed between them. First, we keep identities separate. If both use the local identifier foto, we resolve it together with its originating space: editorial or images. If we import the threads into a shared registry, we assign distinct names and consistently rewrite internal references. We do not merely change thread labels while leaving links pointing to the old names.

We then declare the interface, meaning the information each circuit provides and expects. The points of this interface may be called ports: they are not a new kind of world object, but declared connection points. The editorial circuit provides the file identification, selected version and proposed use. The approvals circuit provides an assessment for that combination and its source. A mapping specifies that the image selected in the article corresponds to the file and version in the other circuit. The photograph's displayed name may help the search, but the adopted mapping relies on verified identification.

The processor then checks interface compatibility: is this the same version, the same kind of use and an assessment still valid for the declared conditions? If approval concerns the previous version, composition produces an unmet condition. It does not automatically transfer the answer to the new file. If the assessment is favorable, the result can unlock the corresponding editorial check; it does not publish the article. The output of composition is a connected collection, with inspectable mappings and conditions either satisfied or still open.

This is an architectural contract, not a hidden effect of the $ sign. SOP can carry the threads and relationships of this description; the application must define the interface's semantics and the operations that check it. Composing a pattern's modules and connecting two independent instances are also different operations. In the first case, we reuse rules and vocabulary; in the second, we establish correspondences between concrete situations. Prefixing names prevents collisions, but does not prove that two representations concern the same thing.

Validation turns a description into a reliable input for calculation

The system may read “a 25-minute transfer” perfectly yet use the transfer between two other places. For this reason, validation is not a single question answered yes or no. It tracks different layers of correctness, each with its own result and its own possibility of repair.

We start from the proposed text, the intended instance and the version of the pattern used. Syntactic reading checks whether the declarations can be reconstructed: an identifier and type exist, values are correctly enclosed in quotation marks, and properties have the expected form. The result is a candidate circuit. An error here indicates a fragment that cannot be read; it is not yet a contradiction about the trip.

Next comes checking identities and references. Each identifier must be unambiguous within its space, and internal links must reach the appropriate threads. If the arrival points to a trip not yet identified, we retain this gap without inventing the object. The pattern can then check types, permitted properties and the number of values: a participant must reference a person, while a unique start time cannot have two incompatible values treated simultaneously as definitive.

Semantic checking of the calculation goes further. The arrival and the meeting’s start must be related to the same temporal convention; the duration must be explicitly converted if the source expresses it in seconds. The name duration_minutes helps the reader but does not perform the conversion. We also check the transfer’s endpoints, the trip date and the condition requested by the user. An absent value leads to an open question. Sufficient data that violate the temporal relationship lead to a negative check result. Incompatible units make the operation inapplicable until the inputs are corrected; they do not prove that the person will be late.

Validation against the pattern does not demonstrate that the data are true. An old timetable may pass every structural check. For this reason, the architecture also retains the source, the time of observation or communication and the information's status. If two sources support different times, the conflict must be located and represented before one is chosen. In a scenario, we can explore each alternative. In the situation used for a command, application policy establishes what evidence is sufficient and what confirmation is still missing.

Acceptance also depends on the requested operation. If we learn that Ana has already confirmed two overlapping meetings, the instance must be able to record the situation and the deviation from the scheduling rule. A prohibition on future confirmations cannot erase what happened. We retain the account with its source; to adopt the conclusion that both confirmations are real, we assess the evidence; for a new confirmation command, we separately check authorization and compatibility conditions. Representing facts, justifying conclusions and accepting actions do not share the same success condition.

The result of validation is therefore a description usable for the declared purpose or a diagnosis showing what limits its use. The parts of a change declared together are checked and adopted coherently, but the instance can remain incomplete. The new arrival can be recorded even if the transfer duration is missing; what remains blocked is the feasibility verdict. Coherent adoption of the change and completeness of the answer are different conditions. SOP syntax, matching and rule mechanisms provide pieces for this construction. Separating sources, scenarios, versions and action authorization belongs to the contract of the system we design around them.

What do we retain from the initial message? The entire check belongs to the eight o'clock train scenario. The system can save that scenario together with the result and present it to the user later. The existing tickets remain represented in the accepted situation

until a change is requested and carried out. The symbolic form lets us track both situations, while dialogue establishes which operation we want on them.

From this point, the representation begins answering questions that go beyond calculating a single option. We can search for another train, change the desired margin or discover what information would let us choose between two routes. Understanding language opened the problem; reasoning can explore the space that the pattern and instance make precise.

4 Reasoning opens possibilities

An invitation to a conference can seem a solved problem the moment we find a train. We arrive in the city at 18:10, the presentation starts at 19:00, and the station is twenty-five minutes from the venue. We have time. Then we read the organizer's message more carefully: speakers must be present no later than 18:45 for the sound check. The itinerary remains possible, but its actual margin is ten minutes. If the preceding meeting prevents us from taking that train, the next arrival, at 18:25, changes the problem. Now it matters whether we can reach the venue in fifteen minutes or need the twenty-five initially estimated.

We started with a sentence, "can I attend?", and reached a collection of relationships between time, travel and obligations. Reasoning begins when we use these relationships to distinguish what is possible from what must be true, to search for a path to a goal or to understand why the plan failed. An SWM makes the situation ac-

cessible to these operations. Each views the same instance from a different angle, and choosing the angle matters as much as the speed of calculation.

A solution found and a necessary conclusion

Let us imagine that the assistant proposes the train arriving at 18:10. The proposal has an easily traceable justification: the arrival, transfer duration and time required by the organizer. We reach the venue at 18:35, before the 18:45 limit. We have demonstrated that a particular itinerary respects the represented conditions. The reader can check the result and immediately see what information would change it. If the transfer takes forty minutes, the same train no longer satisfies the requirement.

The existence of this itinerary does not mean we must choose it. There may be an earlier train, a connection using another means of transport or remote participation accepted by the organizer. A question about possibility is settled by a single verified option. A question about necessity requires knowing whether every permitted option has a particular property. And a question about preference requires something more: what we value in the possible options. Cost, time spent traveling and the risk of delay can lead to different choices, even when all respect the arrival time.

This distinction appears in conversations far more often than logical terminology suggests. “You could leave at four” is a proposal. “You must leave by four” asserts a condition for reaching the goal. An assistant that confuses them may turn the first solution found into an apparent obligation. Symbolic representation provides the place where the exact question can be retained: we are looking for a possibility, checking a necessity or comparing options according to a declared criterion.

Deduction follows the consequences of accepted information. In the trip example, the relationship between arrival and transfer allows the time we reach the venue to be calculated. A constraint ex-

presses what must be respected: that time must be no later than 18:45. The same relationship can also be used in reverse. If we know that the transfer takes twenty-five minutes, we infer that we must arrive at the station no later than 18:20. We have not learned a new rule for this question; we have used the same relationship with different known values.

How does an engine reach these consequences without constantly rereading the entire world? Inside the deduction engine, we can use an agenda of new facts waiting to be processed. In our example, the inputs are the train's arrival, transfer duration and organizer's deadline, each connected to the appropriate trip and event. An index shows which rules use each kind of fact. When the arrival enters the agenda, the engine looks for relevant rules and completes their premises from the instance. Variable names are replaced consistently: the transfer must start from the station of that arrival and lead to the venue of that conference.

After matching, the temporal interpreter checks the units and calculates 18:10 plus twenty-five minutes. It records arrival at the venue at 18:35 together with the rule and premises used. The new fact enters the agenda and may activate comparison with 18:45. If the same fact already existed, we may retain the new justification without introducing it again as new information. The inverse calculation requires an interpreter that knows how to solve the temporal relationship; simply reversing an "if, then" rule would not justify the same result.

An empty agenda means that the rules in this calculation no longer add information: we have reached a fixed point. Termination is guaranteed for this scheme if the set of possible facts and rules is finite, individual matches and operations terminate, and the rules are monotonic, meaning they only add consequences without retracting them. Each fact on the agenda must be processed, and repeated justifications are retained only once as links, without enumerating every possible proof tree. A cycle between rules then does not generate new work indefinitely. If the rules can construct un-

limited new objects or values, finiteness disappears. Rules using the absence of information likewise need separate semantics: information added later can change their result.

Stability of the calculation does not mean we have learned everything. If the transfer duration is missing, the relationship remains unresolved, with the missing premise identified. If two sources give incompatible times for the same arrival, the engine retains the conflict and makes conclusions conditional on the option used, rather than producing a single apparently certain time. The output thus includes justified consequences and still-open questions. A time limit produces an explicitly declared stop, distinct from the fixed point.

We can also leave some values open. If the train arrives at 18:25, the transfer must take at most twenty minutes. This is already a useful answer: it explains why the usual twenty-five-minute transfer is insufficient and specifies what an alternative would have to offer. The system has turned a vague question about attendance into a verifiable condition. When this condition can be passed to a routing service or to the user, reasoning produces a concrete stage of the solution.

When organizing an event, relationships multiply. One guest is available only in the morning, one room permits the technical demonstration, and two speakers use the same equipment. Assigning a time slot to one session restricts the others’ options. Some consequences emerge through constraint propagation without exploring every schedule. If the demonstration can take place only in the room with a projector and that room is free only after lunch, part of the schedule is already determined.

There are, however, situations in which every pair of conditions appears compatible but the whole does not fit. Three sessions, each requiring an entire room, cannot be placed in two nonoverlapping time slots if the same room is the only permitted one. This is why a solver checks the relationships together. When it finds no solution, the result can become intelligible by identifying the conditions that

block one another. The organizer then learns what actual choice must be made: add a time slot, change the room or drop a requirement.

For this problem, the solver's input can be very concrete: one variable for each session's time slot, the list of permitted values and the condition that two sessions do not occupy the room simultaneously. Propagation eliminates options that can no longer respect the conditions. If alternatives remain, the solver provisionally chooses one and continues. An empty domain shows that this choice cannot be completed; the calculation returns to the previous choice and tries another. A complete, checked assignment is a solution. Exhausting every alternative in the finite domain proves incompatibility of the formulated problem; expiration of the search budget leaves the result unknown. Propagation alone can reach a stable state without finding a solution and without proving that one exists.

A plan changes the situation in which it is executed

An article has been written but cannot yet be published. Some claims must be checked against original documents, the illustration awaits the author's confirmation, and the editor wants to see the final form. The request "publish it tomorrow" expresses a goal without specifying all the actions that can make it achievable. A useful plan must identify what is missing, what can be obtained in parallel and what result must exist before the next step. In this case, the order of operations influences the very meaning of "verified article".

Let us establish the publication's rule in the example: the version sent to the public must be the same version for which the required checks and editorial approval have been confirmed. These conditions describe the goal more precisely than the generic label "ready". A verification action can produce a confirmation connected to the examined version or identify a problem requiring a

change to the text. An editing action produces a new version. Publication makes that version accessible to readers. Each verb thus changes an identifiable part of the situation.

Symbolic planning uses descriptions of actions in terms of preconditions and effects. STRIPS made this organization a landmark in the field: a plan is investigated by tracking what becomes true after each step and what conditions are necessary to continue it. [S3] In an SWM, the pattern specifies the meaning of the relevant operations, while the instance shows the article's current version, existing confirmations and open problems. The planner can start from this situation and search for a sequence satisfying the publication conditions.

A planner for the article can retain a frontier of situations not yet explored. Each contains the text version, confirmations, actions that led there and their estimated cost. From a situation, the engine selects operations whose preconditions are met and applies their effects in a working copy. Successor situations result; links to predecessors allow the plan to be reconstructed when the goal is reached. A record of states already analyzed avoids repeating the same searches, provided it includes every difference relevant to continuation, including the version and available time.

Cost can order exploration: we prefer a shorter or cheaper path according to the declared criterion. The first path found is merely feasible if the search method does not also guarantee optimality. For a request for approval, successors must retain acceptance, refusal and absence of a response separately; only an observed confirmation permits success to be adopted in the actual instance. Reaching the goal on the acceptance branch proves only the existence of a favorable evolution. For a plan covering every declared outcome, we check continuation after each response; branches without a continuation remain risks or requests for a decision. The resulting plan describes possible operations and their branches, while execution begins through a separate decision.

Suppose checking the sources discovers an incorrect date and requires changing a paragraph. The revised text becomes the new working version. Previous confirmations must be reassessed according to what they covered: checking an unchanged illustration may remain relevant, whereas approval of the meaning of the changed paragraph must be obtained again. A plan treating all confirmations as properties of the article's title could publish a version that nobody checked. Representing versions makes the dependency between an operation and the exact object on which it was performed visible.

Some activities can advance in parallel. Obtaining the illustration and checking an independent source need not wait for one another. In contrast, approval of the final form depends on integrating the contributions that change it. If an author changes the conclusion while the editor checks an older copy, we do not have two operations that automatically merge into a valid result. The difference between copies must be identified, and we must establish which part of the check can still be used. Useful parallelism follows from independence of effects, not simply from two people being able to work simultaneously.

The plan can also include outcomes outside the system's control. "Ask for the author's approval" is an action the assistant can initiate; "obtain approval" also depends on the response. A realistic description retains the possibility of refusal or a request for changes. It can prepare a version without the illustration, propose postponement or request an editorial decision. Reasoning about the plan thus becomes an exploration of what can be done after each outcome, rather than a sequence assuming that every request will succeed.

Several plans may satisfy the goal. One retains all the materials and publishes later; another reduces the content to the part already checked and meets the initial deadline. Choosing requires criteria that the organization of actions does not invent: which claims are essential, which deadline is firm and which changes the user can approve. The system can show the effect of each choice and identi-

fy the point where it needs a preference or external authority. The plan becomes an explanation of the path to the goal, including decisions still open.

Exploration also has a scope. If we consider only full publication by tomorrow, we may find that no path exists under current conditions. This conclusion does not rule out a shorter edition or later publication. Similarly, stopping the search after a certain time does not prove impossibility. The problem instance can retain the conditions analyzed and the options still open, so that we can continue working after a clarification. When an expected step fails, a new question then appears: what explains the result, and what is worth checking before changing the plan?

A good explanation prepares the next check

Sometimes the meeting was missed even though the itinerary seemed feasible. Now the question has changed. We are no longer seeking a sequence of actions toward a goal, but what might explain an already observed outcome. Was the train delayed, was the transfer duration misestimated, or did the organizer's message concern a different venue? Each hypothesis describes a possible completion of the history. An explanation becomes interesting when we can show what observation would support it and what observation would eliminate it.

This is the point of abduction. We start from an effect and seek premises that, together with the pattern, would make it intelligible. In a home, unusual energy consumption may be compatible with a lower outdoor temperature, a window left open or a change to the heating schedule. The pattern says which relationships are permitted between such conditions and consumption; the instance retains the available observations. The system can organize explanations without arbitrarily selecting one as the actual history.

Temporal information can separate the hypotheses. If the heating schedule changed after the interval during which consumption increased, that change does not by itself explain the earlier increase in the model being analyzed. If the window sensor remained closed, the open-window hypothesis requires either another window or a sensor problem. Such steps bring precision to the investigation. The number of hypotheses may grow, but each extension must retain its connection to the observation it is trying to explain.

Returning to the missed meeting, we can construct an abductive search with explicit inputs: the observation to explain, the accepted facts and a declared family of admissible hypotheses. For each candidate, the engine creates a scenario, adds the hypothesis and checks both compatibility with the facts and whether the rules produce the observed effect. A five-minute delay does not by itself explain missing the deadline in an itinerary with a ten-minute margin. An actual arrival recorded at 18:10 eliminates, in that scenario, the explanation based on a train delay. The remaining candidates are possible explanations; preferring one on grounds of cost or simplicity does not turn it into a fact. If the investigated family has been exhausted and every candidate rejected through checking, the output is “no explanation in this family”. If exploration stops or checking a candidate remains unresolved, we report the incomplete search and the candidates still open. A new observation can eliminate candidates or justify extending the family.

A research example shows the value of this organization even more clearly. Two experiments intended to repeat the same procedure produce different results. The representation can make differences in temperature, sample or instrument version visible. If several explanations remain compatible, the next experiment can be chosen specifically to separate them. A world instance thus becomes a support for investigation: it not only records the result but retains open questions and their dependence on evidence.

There is a difference between explaining a result and predicting the effect of an intervention. Observing high consumption on cold days may help a forecast. To decide what happens if we change the

heating schedule, we need a model of that change's effect under the relevant conditions. A simple association between two values does not specify what remains fixed and what changes when we intervene. Similarly, solving an equation in reverse may give us a missing value without identifying the mechanism that produced it.

Deduction, planning and the search for an explanation can cooperate in the same conversation. The system deduces that the initial itinerary no longer satisfies the required time, searches for an alternative plan and proposes a check on the cause of the delay. Each result retains its role: a consequence of premises, an explored possibility or a hypothesis about an event. When new information appears, these roles also tell us what must be revised. A world model becomes truly useful when it can continue reasoning after the situation has forced it to change its answer.

5 A world that can change its mind

A report on an educational program states that the method was tried in five schools. The team preparing to expand the project uses this information to support the claim that the experience covers several contexts. Then a correction appears: it was five classes in the same school. The observed results may remain important, but one reason invoked for expansion must be reexamined. Correcting the word "schools" is insufficient if the project presentation and final recommendation continue to rely on diversity of locations that were not studied.

For an assistant working with documents, the difficulty is tracing information through to its consequences. Which statement was corrected, where was it used and which conclusions still have support? An SWM can organize this trace through explicit objects, sources and dependencies. Memory becomes capable of retaining the value of observations while revising an interpretation that exceeded them. The representation's power appears in how it makes the difference between yesterday's answer and today's intelligible.

The same result can have two different histories

In the current file, the program appears in a single school. This result may come from correcting the initial report: only one institution had participated from the start, with several classes. But the same current value could also appear in another project in which five schools initially participated and four subsequently withdrew. In the first case, we correct our knowledge about the activity. In the second, we record an event that changed the activity. A field containing the current number of schools cannot, by itself, distinguish the two histories.

The difference influences what we can claim about the experience accumulated. If participation had been reported incorrectly, the conclusion about several contexts loses that premise. If schools withdrew after participating, observations collected in them may continue to support claims about that period, under the study's conditions. At the same time, reasons for withdrawal may become relevant information for continuing the project. The instance must connect data to the appropriate institutions and times so that a change in the current situation neither erases nor invents the past.

An earlier decision must also be understood in relation to the information available then. We can retain the report's old wording, the fact that it was used in a recommendation and the correction that replaced it. The new analysis can show that the recommendation had an incorrect premise without presenting the situation as

though the assistant had known the correct information from the start. This trace is useful for improving the process: it can show where a source should have been checked or where ambiguous wording was turned into an overly strong claim.

To perform this tracking, we can give each statement its own identity. Its record specifies the object we are discussing, the property or relationship asserted, the value and the interval described. It then connects the content to the source, the time we received it and the instance in which we use it. “Five schools participated last semester” and “one school participates now” can coexist without conflict. By contrast, two different numbers assigned to the same participation, the same period and the same definition of a participating school require checking. Comparison begins with this identification of the shared question.

The record also indicates the statement’s role: observation, account from a document, estimate or working hypothesis. It specifies whether we use it in the adopted situation or an explored scenario. An estimate that five schools will participate and the report that one participated are not two competing measurements; their difference may be precisely the result we want to explain.

The source and the adoption of the statement are retained separately. The report may continue to contain the number five even though the system no longer uses it as a premise for the current recommendation. Upon correction, we connect the new record to the statement it replaces and the period corrected. We end use of the old value for that analysis while retaining the history of its adoption. We can thus answer both “what do we now know about last semester?” and “what information was the recommendation at that time based on?”. A retrospective update changes the first answer without rewriting the second.

In conversation, verbs and context often provide the decisive clue. “We reported it incorrectly”, “they withdrew”, “we only just learned” and “they remained” may require different operations on the representation. The neural component proposes the message’s

interpretation, while the pattern establishes the meaning of a correction, a new event or a hypothesis. If the remark “only one school remains” does not allow a choice, a local question can prevent a false history: has participation changed, or are we correcting the record we had?

The order in which documents arrive can create similar confusion. An appendix written before the correction may arrive today, after the corrected report version. If the system automatically adopts the last file received, it may reintroduce the error just eliminated. We must know when the document was written, what period it describes and whether it has been superseded for the statement we use. An old document can remain the appropriate source for describing a past stage even if another represents the current situation. Information’s currency is established in relation to the question, not merely by the file’s position in the message list.

We can also see why a current value is not the entire instance. A project currently “in progress” may come from a normal beginning or from reopening work considered finished. In the second case, there may be checking obligations, people who must be informed and conclusions that depended on completion. The instance represents the case’s continuity, including the history relevant to what we can do next. What we retain of that history is a modeling decision guided by questions, not an obligation to archive every possible detail.

Conclusions live through the connections that support them

In a report about preparing an event, we read: “The room can be used for an online broadcast.” The conclusion rests on the existence of a suitable network connection and the necessary equipment. Later, we learn that the network will be shut down for maintenance. The report cannot be correctly updated merely by chan-

ging the connection's state. We must find the conclusion about broadcasting, the plan that used it and the messages that presented it as a confirmed option.

A justification makes these connections visible. It can say that a conclusion was obtained from particular information through a particular rule. Truth-maintenance systems have studied precisely the relationship between premises, conclusions and revision; Jon Doyle's work is a landmark in this direction. The useful idea for SWM is that an explanation can also serve as a structure for updating. When a premise changes, we have a starting point for identifying affected consequences. [S4]

In our design, the justification receives its own record: it indicates the rule version, the statements used as premises and the resulting conclusion. For broadcasting, one justification may require the fixed connection and suitable equipment together; another may require the mobile connection and the same equipment. The premises within a justification must be satisfied together, while justifications offer alternative paths. A reverse index allows rapid retrieval of every justification using a particular statement. Ordinary references between objects do not replace this index: a room having an address does not mean every conclusion about the room depends on its address.

The situation becomes interesting when there are several paths of support. The room may permit broadcasting through its own connection or through available, suitable mobile equipment. Withdrawing the first possibility does not automatically cancel the general conclusion. It should withdraw only the justification based on that possibility, then check whether the alternative remains valid. A system that mechanically deletes every conclusion connected to the first premise may lose results still supported. A system that retracts nothing may retain results left without support.

We can trace the retraction in practice. The maintenance event changes the connection's availability for the interval concerned. The index sends the broadcast's justification to a reevaluation

agenda. The interpreter checks its premises and marks that path as unavailable. It then consults the conclusion's other justifications: if the mobile equipment remains available and suitable, the conclusion survives with a changed explanation. If it loses every path, it is retracted from adopted conclusions, and its dependents in turn enter the agenda. The process outputs the difference between the previous situation and the revised one: conclusions retained, retracted or still to be checked, each with its reason.

Nor does a circle of statements provide an independent basis merely by existing. If one document says the event is confirmed because the room is reserved, while another says the room is reserved because the event appears confirmed, we must find the act or message that actually established the reservation. Links can describe information's circulation without showing where it originated. Provenance and justification work together: the first tracks sources and transformations; the second shows how information supports a conclusion.

A simple justification counter would miss this case: each statement would still have support in the other. For rules that add conclusions from positive premises within a finite domain of possible statements, one clear solution is to reevaluate the subgraph affected by the retraction. We identify it by following the index from the retracted premise to justifications and their conclusions, without visiting the same identity twice. We suspend support derived from within it, retain still-adopted base premises and valid external support, then reconstruct consequences until nothing new appears. A baseless cycle does not reactivate itself. We publish changes after this stabilization. Rules using the absence of information additionally need explicit conditions for a complete record; the same procedure does not automatically establish their meaning.

Conflicts between sources sometimes require clarifying meaning before choosing a winner. A show's page announces 18:00, while the ticket indicates 19:00. This may be an error, but it may also be the difference between doors opening and the show starting. If the model retains only the generic property "event time", two compat-

ible pieces of information may appear contradictory. Introducing the two temporal roles changes the pattern justifiably: it makes room for a distinction we need to interpret both sources correctly.

Other conflicts are real. Two confirmations may indicate different times for the same session. The instance can retain both accounts, with author, time and the object they concern, while the application requests confirmation or applies an explicit authority rule. Even then, the answer may depend on the purpose. For choosing the day of travel, the difference between times may be irrelevant. For reserving a transfer, it must be resolved or retained as alternatives.

In evaluation, absence of support and existence of incompatible support must produce distinct results. If the time is unknown, the transfer check remains waiting for a premise. If two disputed times exist, we can calculate two alternatives while keeping the conflict visible. If a rule concerns another kind of event, it is inapplicable to that case. None of these results means the transfer has been demonstrated impossible. The interpreter returns both the check's state and the statement, conflict or applicability condition explaining the stop.

For the statement "the connection is available during the event interval", we can separately retain favorable support and explicit opposing support. Absence of both leaves the answer unknown; presence of both opens a conflict to resolve. Evidence of unavailability differs from absence of confirmation. The decision rule specifies what it can use from each situation, and a local conflict does not authorize unrelated conclusions.

A system capable of maintaining such differences need not forcibly make all its information uniform. It can say what is adopted for a particular operation, what remains disputed and which conclusions are shared by every alternative. Coherence comes from correctly representing roles and dependencies. An account of a scheduling violation can be retained precisely to explain why reality departed from the plan.

There is another situation that can easily go unnoticed: information is missing. The documents received contain no confirmation from a guest. If we are working with a partial collection of messages, absence may mean the reply arrived through another channel. If the registration system is the designated source for recording every confirmation, a missing entry has a precise operational meaning within that system. The pattern must show under what conditions the record is considered complete for the question asked. The application can thus decide to provisionally retain the place, send a reminder or request checking another channel without automatically turning silence into refusal.

Here we observe a difference between knowledge and policy. The organizer may establish that badges are printed only for people confirmed in the registry. The rule permits a decision even when we do not know every guest's intentions. It does not demonstrate that people absent from the registry will not attend. When one of them appears at the event, the instance must be able to record the arrival and open the practical question of the badge. A description keeping these roles separate remains useful precisely when the real situation exceeds what had been prepared.

Confidence becomes useful when we know which question it measures

The forecast says there is a probability of rain, and the organizer must decide whether to move the event indoors. A percentage may seem the simplest form of answer. Yet the decision depends on what that percentage describes: rain during which interval, in which place and above what intensity? For a twenty-minute meeting and a three-hour concert, the same forecast can have different implications. The instance must connect the estimate to the event the user is asking about.

Before probability, there are also other forms of not knowing. We may not yet have identified the event's location. We may know two possible dates without having reason to assign equal chances to each. We may have a temperature measured with a particular tolerance. We may have a statistical estimate of a train's delay. These representations require different operations. A list of alternatives says which options we retain; an interval bounds values; a probability distribution adds information about how outcomes are weighted.

Confidence in formalization also concerns a different question from confidence in content. A language model may confidently recognize that the message "I'll probably arrive at six" refers to the arrival time. This success does not turn arrival at six into a certain fact. The system has correctly identified an estimate expressed by a person. For reasoning, both the message's meaning and the uncertainty it communicates must be retained.

Dependence between sources is just as important. Three websites may display the same time because they copied information from a single announcement. Counting them as three independent confirmations would artificially increase confidence. In document analysis, provenance allows this shared origin to be recognized. In an observation system, two indicators may come from the same sensor. How many sources we have and how many independent observations we have are distinct questions with direct effects on probabilistic calculations.

For this reason, an estimate retained in the instance must indicate which event it measures, the model that produced it and the conditions used. When a condition changes, we send the estimate for recalculation or mark it as outdated. Combining two estimates requires an interpreter that knows the relationship between the events and the data's dependence. If this relationship is missing, we can retain the estimates separately and explain what we cannot yet combine. Adding confidence scores cannot replace the missing probabilistic model.

A simple example can show when a probabilistic number is well defined. We assume the only uncertainty relevant to attending a meeting is whether the train arrives no later than 18:20. A model evaluated for that context assigns the event probability 0.8, while the twenty-five-minute transfer is treated as fixed. If being present by 18:45 is possible exactly when this arrival condition is met, the modeled probability of success is 0.8. If we add uncertainty about the transfer or a change of venue, the model must also be extended, because we have changed the event whose probability we calculate.

Evaluating these estimates is empirical. When the system assigns 0.8 to many comparable events, we expect them to occur in approximately the indicated proportion. This is the idea of calibration. It connects a number to observed behavior in a relevant domain and over a relevant period. A score produced by a model becomes information for a decision through its definition and evaluation, not through the number of decimal places displayed.

The final decision also depends on consequences and preferences. A person may accept a risk of delay for a visit without a fixed time and prefer an earlier departure before a presentation. SWM can retain the chosen criterion and explain which estimates were used. When a new forecast appears or the goal changes, the system can return to the appropriate decision. Sometimes the available information is already sufficient; at other times, a detail separating two choices is missing. Recognizing this difference is the beginning of a dialogue worth having.

6 The right question at the right moment

“Choose a room where we can all fit.” The assistant could immediately ask for the number of participants, time, budget, desired equipment, transport access and preferences for natural light. All may be useful information. Yet if the only urgent choice is between a twenty-seat room and a fifty-seat one, and the user has already said at least thirty people will attend, we can already eliminate the small room. For the fifty-seat room, we still need to check the group’s upper limit, but the conversation has advanced: an option has been excluded using existing information.

An assistant’s value also appears in what it does not force us to explain again. For this, the system must retain the goal, the information already established and the relationship between unknowns and remaining options. An SWM can make these connections inspectable. The right question is not necessarily the one about the emptiest field, but the one whose answer lets the conversation advance in the direction requested by the user.

The detail that separates two decisions

We are organizing an evening screening followed by a discussion. One room is available until 21:00, another until 23:00. The film lasts ninety minutes, and the proposed start is 19:00. If the user wants only the screening, the first room may suffice. If they also want an hour-long discussion, the same room no longer covers the program. The discussion’s duration has become an unknown that changes the choice. The question “How much time would you like to keep for discussion?” arises from the situation’s structure, not from a fixed list of mandatory fields.

We can follow the reasoning step by step. The instance retains the start time, film duration, room intervals and the fact that an activity follows the film. The pattern says how successive activities are connected and how we check whether they fit within an interval. The system explores relevant values for the still-unknown duration. With twenty minutes of discussion, the end is at 20:50; with sixty minutes, at 21:30. A condition is satisfied in one option and violated in the other. The question thus has a justification the assistant can express simply: using the room available until nine depends on how long the discussion lasts.

How do we turn this observation into a mechanism for choosing questions? We start from the operation requested now: we want to know whether we can keep the room available until 21:00. A message such as “let’s also have a short discussion” may lead to several candidate interpretations. They retain the expression’s source, the activity it concerns and the durations compatible with the available information. A neural model’s proposal remains a candidate until the instance’s conditions or an answer support it; the system can also retain the fact that the interpretation list is still incomplete.

Suppose the remaining plausible interval is twenty to sixty minutes. The interpreter checks the options for these alternatives and groups them by their answer to the current question. The film ends at 20:30, so any duration of at most thirty minutes keeps the first room feasible; a longer duration excludes it. We need not simulate every minute separately. The temporal condition gives us the threshold separating two groups. Under the conditions analyzed, the room available until 23:00 can accommodate both groups. If the user only wants a feasible room and accepts this second option, the exact duration may no longer be necessary for the choice.

Candidate questions are constructed from differences separating these results. “Will the discussion exceed thirty minutes?” is sufficient for the first room; “will it last twenty or twenty-five?” separates two situations with the same result and does not help that decision. Grouping thus depends on the goal. If the exact finishing time must be published, twenty and twenty-five become different

answers, and the more precise clarification gains value. We retain both the alternatives and the question for which we considered them equivalent.

Not every question needs to come with the full proof. The user generally needs the local reason for clarification. “Would you like to keep an hour for discussion? In that case, we need the room available later.” The explanation shows why the answer matters and allows the goal to be corrected. The user may say the discussion is optional or that the event can start at 18:30. Clarification does not merely fill a value; it can change how the problem is formulated.

The cost of information matters. The approximate number of guests may be known immediately, while confirmation from every participant may take several days. If both plausible values lead to the same room, we can advance without the exact number. If options change at a nearby threshold, the estimate becomes important. A system can choose to consult an existing record, retain a buffer or ask the user for a decision about risk. Asking a question is one available action for reducing uncertainty.

To compare clarifications, we estimate how much difference their answers would make and how much effort they require. If we have justified probabilities for the answers, we can calculate the expected reduction in a defined loss, such as choosing an unsuitable room. Without them, we can use a simpler declared criterion: prefer the accessible question that separates alternatives with the most important consequences. We do not automatically assign equal chances to the options. Stopping also has a condition: at least one interpretation remains and all permit the same answer to the current request, the user has chosen a conditional option, or the necessary information cannot be obtained now. If the interpretations cover only part of the possibilities still admitted, the answer remains conditional on that coverage. When decisive information is missing, we return the options and the unresolved condition. Eliminating all candidates requires clarification or extending the search; it does not produce a shared answer. Exhausting search time does not prove that we have found every interpretation.

Some clarifications also prevent acting on the wrong object. “Move it to Friday” may concern the meeting, the trip or the room reservation. If the current discussion and recent references unambiguously identify the meeting, an additional question may be unnecessary. If two objects remain plausible and the change produces external effects, the difference must be resolved before the command. The pattern and instance provide additional interpretive criteria: a room is reserved for an interval, a trip has a departure and arrival, and a meeting has participants who must be informed.

Sometimes all interpretations lead to the same requested information even if the reference is not fully resolved. If the user only asks whether Friday is a working day in the project’s already established calendar, answering that question may suffice. If they ask us to send the invitation, identifying the event becomes mandatory. An interpretation’s sufficiency depends on the operation. Efficient dialogue retains what has been clarified and recognizes when a new action requires greater precision.

Context continues to exist when we change the subject

In the same conversation, we may discuss the event, then a trip and then a text we want to publish. When we return with “for Friday, we’ll keep the earlier option”, the intention relies on context that may not be in the last two messages. A chronological summary helps, but is insufficient if we do not know which options were still open, what had been chosen and what changed in the meantime. For continuity, we need access to the situations discussed, with their relevant state.

An implementation can maintain a registry of discussed instances, searchable by objects, participants, goal and references to messages. Each entry indicates the pattern version used, the adopted scenario, retained alternatives and remaining questions. On return, the system finds instances compatible with the remark and

checks references; if two remain plausible for the requested operation, it clarifies identity. After identification, it revisits justifications affected by information received in the meantime. “The earlier option” may be correctly retrieved yet no longer be feasible.

An answer to a question also needs an address. When asking about the discussion’s duration, we retain the connection to the event, scenario and state in which the question was formulated. If the answer “twenty minutes” arrives after the screening has been moved, we apply it to the identified activity and recalculate in the new situation. We do not automatically reactivate the old room choice. Dialogue memory thus becomes a mechanism for resuming work, including when clarification and changes in the world occur in parallel.

Conversation topics may use different worlds or concern parts of the same world. Organizing an event and publishing its invitation share some identities and dates but have different operations. In the first discussion, we choose an interval and a room. In the second, we check wording, approved information and publication time. Confirming a text does not automatically confirm the room reservation. Conversely, reserving the room does not establish that all information can be published. Connected instances retain together what is shared and separately what has a different role.

A change of subject must not freeze reality. While we discuss the text, the room may become unavailable. When we return to organization, the assistant should be able to say that the choice discussed was valid then but has lost one of its conditions. Resuming dialogue thus requires two recoveries: where we left off in deliberation and the current state of the things it depended on. This distinction becomes important in applications used over several days, where conversation and actual activity follow different rhythms.

Research on grounding in communication, associated with Herbert Clark and Susan Brennan, concerns how participants establish shared understanding sufficient for the conversation’s purpose. This idea provides a useful reference for assistants: adopted mean-

ing is constructed through contributions and confirmations under the conditions of the activity. [S15] In an application, we can track which interpretation was proposed, what correction the user gave and what information became a basis for the next step.

For example, the assistant may say: “I understood that we’re moving the discussion after the film, not the screening.” The user replies: “We’re moving both.” This remark does not merely add a fact about two activities; it corrects the operation’s scope. If the system retains the connection between message and interpretation, it can change both elements and reevaluate their dependencies. If it retains only an instruction’s final form, it risks forgetting what was corrected and why other conclusions need updating.

In a conversation with several participants, things become even more interesting. The organizer confirming the new time does not prove that guests have received it. An SWM can represent confirmation of the program and the state of communication to participants as different relationships. Thus, the question “do we still have anything to do?” can receive an answer about outstanding invitations even when the program itself is settled. Knowing something about the event and knowing that the others have been informed are distinct capacities of shared context.

Good continuity appears in very concrete details. The system resumes options still under discussion, uses the appropriate identities and checks assumptions that have become outdated. The user need not recite the entire conversation to continue. In this sense, world representation is also an instrument of cooperation: it retains what participants can rely on and makes visible the differences worth bringing back into dialogue.

Dialogue can also change how we describe the world

The user says the room is free until 21:00. Later, they add that the last thirty minutes must be kept for cleaning. Until then, the simple pattern treated closing time as the limit until which the public could remain. We now discover a necessary distinction: the end of the activity with the public and the end of room use. A new value in an old field would hide this difference. The pattern must also be able to represent the time needed after the event.

This is a form of learning through dialogue. A concrete case shows that the reusable description did not separate two situations that must produce different answers. We can propose an extension through the cleaning activity and its relationship to the schedule. We then check it on other cases: some rooms include the service, others require complete clearance, and some events need additional preparation before public access. The pattern becomes more expressive for an identified need without turning every observation into a universal rule.

The new distinction also changes the answer at the chapter's beginning. The film starting at 19:00 ends at 20:30. If the room must be vacated at 21:00 and cleaning occupies the final thirty minutes, no time remains for the twenty-minute discussion we considered feasible. The assistant can explain exactly why the conclusion changed and propose an earlier start or the room available until 23:00. Improving the pattern has produced a consequence in the instance, and dialogue can continue from that consequence without hiding the premise missing from the first analysis.

The information's authority matters too. "I prefer us to have half an hour" expresses the organizer's preference. "This room's contract specifies half an hour" describes a condition of the case. "All our events keep at least half an hour" may express an organizational rule. Numerical similarity does not make them equivalent. In

dialogue, a question about a statement's meaning may be more important than the value's precision. The instance and pattern must be modified in the appropriate place.

The assistant thus has two activities that influence one another. One tracks the situation: what information is old, which reference is ambiguous, which condition is missing. The other chooses what is worth doing next: a question, a check, exploring an option or executing a command. The distinction between monitoring and control, formulated by Nelson and Narens in a framework on metamemory, offers a point of comparison for this organization. [S16] In the SWM architecture, the two can be evaluated separately: the system may correctly detect an uncertainty yet choose a question of little use.

We can observe this in a dialogue where participant numbers are still approximate but every remaining room can accommodate them. Detecting the unknown is correct. Insisting on the exact number, however, may delay selecting decisive information, such as access to a projector. A good control mechanism tracks the clarification's effect on the current goal and the cost of obtaining it. It can also decide that the moment has come to present two conditional options, leaving the user a choice based on preferences.

A mature conversation can remain useful even when some matters remain unsettled. If we do not know when a preceding meeting ends, we can prepare two itineraries and establish which information will select one of them. If the duration of the discussion after the film remains flexible, we can describe the program that fits in each room. The instance retains this decision structure, and returning later no longer requires reconstructing it from text fragments.

At this point, questions are no longer merely instruments for collecting data. They participate in constructing a shared representation, revising the rules used and choosing actions. After the user says "we'll choose this option", a new responsibility appears: the operation to be executed must preserve the meaning of the ex-

plored option. Representation and dialogue have brought us to an intelligible decision. The next step is to investigate the action itself before it changes the world being tracked.

7 Actions we can investigate before executing them

On the evening before a conference, one guest announces that they will arrive later. The organizer asks the assistant to move the program by ten minutes. On screen, the change seems modest: the same people, presentations and breaks. Yet the final contribution would end after the time when the room must be vacated. A system faithfully executing the addition can produce a program that is impossible to follow.

The situation contains two questions whose answers tell us different things about the same action. Which properties does the operation “move the entire program by ten minutes” preserve? Under what conditions can that operation be applied in this case? The first question concerns the form of the transformation. The second concerns the transformation’s relationship to the world in which we want to use it. When we describe an action precisely enough, we can investigate both questions before changing any reservation or sending guests a message.

This possibility changes a symbolic model’s role. The world pattern becomes a place where actions can be examined, compared and corrected. The concrete instance supplies data for a check, but the same description can be used for an entire family of situations.

Here we encounter symbolic execution and algebraic interpretation: techniques that let us ask what a program does before entrusting it with changing a world.

What remains the same when we move a program

In the conference program, the first presentation occupies 9:00–9:40 and the second 9:45–10:15. The room must be vacated at 10:20. If we shift both intervals by ten minutes, we obtain 9:10–9:50 and 9:55–10:25. The break between presentations is still five minutes, and their durations remain forty and thirty minutes. The arithmetic is correct. The mismatch appears where it meets another condition: the right to use the room ends at 10:20.

To understand why some relationships were preserved, we can temporarily set aside the concrete times. We call a presentation's start s, its end e and the requested shift d. All are expressed in minutes relative to the same temporal reference. The action produces a start s + d and an end e + d. The duration after the move is calculated as follows:

$$\text{new duration} = (e + d) - (s + d) = e - s$$

The shift cancels out of the expression. We have shown that duration remains the same for every value for which we defined the operation, not only for the conference example. We can do the same for the break between two presentations. If the first ends at e_1 and the second begins at s_2, the initial break is $s_2 - e_1$. After the shared shift, it becomes $(s_2 + d) - (e_1 + d)$, meaning the same difference. Consequently, a sufficient break remains sufficient when we shift both events in the same way.

This is an algebraic interpretation of the action. Rather than immediately replacing every symbol with numbers, the interpreter constructs expressions and applies rules to them. A property of the result becomes a relationship we can simplify. The operation is especially useful when we want to check a reusable rule: the same res-

cheduling function will be used for many conferences, classes or meetings, and an error in its description would recur in every instance.

A small program defect might change only the start time, leaving the end unchanged. In that case, the resulting duration would be e − (s + d), meaning the initial duration minus the shift. For the first presentation, a ten-minute delay would turn forty minutes into thirty. Symbolic analysis makes the error visible before choosing an example. The numerical example then translates it into a consequence the organizer immediately understands: the guest has lost a quarter of their time.

Within the pattern, however, moving a program has more obligations than preserving duration. Events must remain within the room's available interval, participants must be able to attend, and a scheduled broadcast may have its own timetable. Algebra helps precisely because it allows these aspects to be separated. Internal relationships preserved by the transformation can be proved once for the analyzed fragment. Conditions depending on the concrete case are checked on the current instance.

We can even determine the maximum shift the room permits. The final presentation initially ends at 10:15, and the room is available until 10:20. The condition "the new end is no later than 10:20" implies that the shift can be at most five minutes. The system can answer: "Ten minutes exceeds the room reservation; five minutes preserves this condition. For a larger move, the reservation must be extended or the program's duration changed." The explanation offers a real intervention option connected to the exact reason for the blockage.

The checked property has a precise domain. We worked with durations and moments on the same temporal axis. If we change a meeting's day using local times, a transition to daylight saving time may require an additional time-zone rule. This is an example of choosing a representation according to the question: elapsed

minutes and the time displayed on a clock have different relationships. The appropriate pattern makes them explicit where they may change the result.

The paths a program can follow

Suppose the organizer accepts a five-minute move, but the room is managed by an external service. A hasty implementation might cancel the old reservation and only then try to create the new one. If someone else takes the desired interval in the meantime, the organizer is left without either option. The program’s arithmetic contains no error; the mistake lies in the order of operations and the handling of a refusal.

Symbolic execution allows such paths through a program to be examined. Rather than running with a single set of concrete responses, the interpreter retains symbols for values still unknown and separates paths at conditions. Each path is accompanied by the premises that make it possible. The idea appears in James C. King’s classic work on symbolic execution and program testing. [S12] For our example, a success response from the reservation service leads along one path, while a refusal leads along another.

To construct this interpreter, we must know what a path carries with it. We propose an analysis state retaining the current program point, the variables’ symbolic values, the modeled objects and the path condition. This last expression combines every requirement that must be true to reach that point. In an analysis of shift requests, the shift d may start with a domain from zero to ten minutes. After updating the program, the final presentation’s end is the expression “10:15 plus d”; the state retains the expression without yet choosing a value for d.

At the room-availability test, the interpreter creates two states. The first adds to the path condition the requirement that d be at most five; the other adds the requirement that d be greater than five. Each copy continues with its branch’s instructions. If the first

branch later contained a test requiring d greater than seven, that continuation would simultaneously impose d at most five and d greater than seven. A satisfiability checker can establish that no value respects both conditions, so the interpreter eliminates the impossible continuation.

We can trace the defect step by step. Initially, there is a confirmed reservation. The program cancels it, so in the intermediate state we no longer have that commitment. It then requests the new interval. On the path where the request is refused, the program ends without a reservation. This path satisfies the described conditions: the initial reservation existed, cancellation succeeded and the new interval was unavailable. Yet it contradicts the desired property: an unsuccessful rescheduling must preserve the initial reservation.

The analysis result can be presented as a counterexample with concrete states and responses. For a developer, it shows the branch that must change. For a pattern designer, it shows that the verb “reschedule” hides a richer commitment than two successive instructions. For the user, it justifies intelligible behavior: the assistant retains the confirmed option until it can establish which replacement is possible under the conditions of the service used.

The counterexample can be sought through a precise question addressed to the solver: do values exist that satisfy the path condition and violate the required property at the output? In the reservation example, the values include responses from external operations. The combination “cancellation accepted, new reservation refused” leaves the initial reservation absent and satisfies this question. The solver can provide the values that make the error feasible within the model. If the real service does not permit that combination, its contract must explicitly exclude it. An imaginary service response cannot become an executable counterexample merely because we have called it a symbol.

The checker can also answer that the problem remains unknown, for example when operations exceed the fragment it can decide or computation time expires. The interpreter then retains the branch

as unresolved. A branch ends when the program reaches an exit, an error or the explicit exploration limit. If the program has loops or numerous interactions, the number of paths may grow without a useful bound. The analysis then delivers the paths checked, counterexamples found and what remains unexplored. Absence of a counterexample in this collection is not a proof covering every execution.

One solution can use the atomic modification operation provided by the service, if one exists and has the required contract. In another system, a temporary reservation of the new slot may be needed, followed by confirmation and release of the old one. The choice depends on the available operations and the possible costs of a period with two reservations. The symbolic model can describe these possibilities, while the integration code must follow the chosen protocol.

What a branch represents thus becomes important. “We do not know the availability” is a state of our information. “The service rejected the slot” is the result of an operation. If the program treats them alike, it may cancel a plan too soon or present an option that has not yet been checked as impossible. A pattern that distinguishes these situations enables a program to ask, wait or seek an alternative without distorting what it knows.

At each step, conditions must be evaluated in the state produced by the previous step. If two organizers try to occupy the last available slot simultaneously, both may initially see that it is free. Analyzing an isolated instance explains each intention, but enforcing the shared rule also requires a coordination mechanism in the service that owns the calendar. At this point, a property of the representation meets a property of distributed execution. Their relationship must be described; otherwise the same sentence, “the slot is free,” would be used with two different temporal meanings.

A description that can be computed in several ways

Before publishing the schedule, the organizer learns that the guest will be between five and ten minutes late. An interval analysis can immediately show that the last presentation would end between 10:20 and 10:25. We now have a more informative answer than a single verdict: a shift of five minutes satisfies the room's limit, while larger values exceed it. Clarifying the exact delay matters to the decision because the remaining alternatives lead to different results.

This form of calculation illustrates abstract interpretation: we analyze sets of executions through more compact representations of their properties. Patrick and Radhia Cousot formulated a framework for relating such approximations to program semantics. [S29] In an analysis intended to support verification, the calculated interval must include all relevant concrete results. If the entire interval satisfies a condition, we can use the result for that condition. If the interval crosses the boundary between acceptance and rejection, we need more information or a more precise analysis.

Compressing information can lose useful relationships. Consider a presentation lasting thirty minutes whose start is shifted by a value that is still unknown. Its start and end depend on the same shift. If we retain only a separate interval for each, without the shared dependency, the calculation may suggest several durations. The algebraic expression preserves the relationship and shows that the duration remains exactly thirty. Representations are chosen according to the property we want to establish: an interval for the end time may suffice, whereas preserving duration requires a relationship between two times.

We can see the precise loss in the second presentation. For d between five and ten minutes, the start is between 9:50 and 9:55, and the end between 10:20 and 10:25. Subtracting the intervals independently gives durations between twenty-five and thirty-five minutes. This interval includes the actual duration, but also allows

the impossible combination of the maximum shift of the start and the minimum shift of the end. If such an approximation signals an unsuitable duration, we return to the shared relationship with d before declaring that we have found a defect. The abstract alarm is a request to refine the analysis.

In an SWM, the same action can be examined through these complementary interpretations. With concrete values, we calculate a proposed schedule. With expressions, we check which relationships the transformation preserves. With intervals, we analyze the effect of uncertainty. When the user adopts the option, the executor can begin operations on the real service. The pattern specifies the action's meaning, and each interpreter declares what it can calculate from that description.

This requires discipline in design. If the simulator adds five minutes to both ends of an interval while the executor updates only the start, the analysis has examined a different operation from the one executed. A shared description of effects, a well-defined fragment of expressions and verification of the implementations that interpret it reduce this gap. SOP can carry the declarations and the links between them; the component that understands an addition, a condition or a calendar command performs the corresponding calculation or effect.

The shared contract for shifting can specify the input as an identified schedule and a shift in minutes, with times expressed on the same timeline. The operation transforms both ends of each presentation and preserves its identity; preconditions concern the value domain and applicable external conditions. The concrete interpreter produces times. The symbolic interpreter produces expressions and path conditions. The abstract interpreter produces a coverage of possible results, using operators that include the concrete effects of the same transformation. If the implementation uses numbers with bounds or rounding, these become part of the semantics the analysis must cover.

At the boundary, the executor receives the proposed schedule and the reservation version on which it must operate. Its response must distinguish a confirmed change, a rejection and an outcome that is still unknown. The calculated times remain a proposal until the effect is confirmed. Interpreter equivalence does not follow from the operation's shared name: each implementation must be checked against its contract. We can thus change the analysis method without secretly changing the question about what the action does.

We therefore use symbols to make actions inspectable. Some properties can be proved for an entire family of cases, others require information about the present instance, and others become known by observing the external result. In everyday life, much of this information comes from images, sounds and continuous processes. The next question is how we bring these observations and learned regularities into the same architecture, preserving the link between what the system recognizes and what it can explain.

8 Several representations of the same world

A robot sees a cup on the edge of a table. To move it, the robot must estimate where it is, how it can be grasped and what will happen if its arm moves in a particular way. If we ask it to take that cup to the room where a meeting is starting, another family of questions arises. Which meeting? Which room is it taking place in now, after the schedule change? Is this the cup prepared for the guest or the one someone is still using? Moving the arm and interpreting the request belong to the same activity, but require representations with different properties.

The term world model covers several responses to this need to anticipate and organize a situation. Some models learn regularities from images and the effects of actions. Others express physical laws. Others make the objects, relationships and conditions of a decision explicit. The interest of the SWM architecture grows precisely where these responses must cooperate: an estimate becomes information about an object, an intention becomes a goal, and a permitted action must be transformed into behavior that succeeds.

A future learned from experience

A child can anticipate that a ball will keep rolling without knowing the equation of motion. Similarly, an artificial system can learn to predict aspects of an environment's evolution without its designer enumerating every relationship. Previous observations and actions become the material from which a representation useful for prediction is constructed. When the representation retains the regularities the agent needs, the agent can compare behaviors before actually trying them in the environment.

In World Models, David Ha and Jürgen Schmidhuber explored an architecture that compresses visual observations and models the evolution of the resulting representation. A controller uses the information provided by the model to choose actions. The experiments include training a behavior in an environment generated by the model and transferring it to the task environment. [S7] The idea that interests us is the possibility of separating the learning of an environmental representation from the learning of the behavior that uses it.

This separation can be understood through a racing game. The agent needs to anticipate how the image of the road changes when it steers and which situations lead it off the track. A useful internal representation can encode these regularities without having fields named “curve,” “grip” and “track edge.” Its coordinates are chosen

through the learning process. We say the representation is latent because its content is not provided directly as objects and relationships with names established by the designer.

DreamerV3 develops the use of a learned environmental model to improve behavior through imagined trajectories. The study published by Hafner and colleagues in 2025 examines numerous control tasks with the same algorithm configuration. [S8] For a system builder, the important point is that a model of change can be a reusable resource in the process of learning actions. The agent benefits from estimated consequences even if they are not presented as a chain of rules intelligible to a reader.

Prediction does not always have to reconstruct every pixel of a future image. V-JEPA 2 pursues prediction in a space of representations learned from visual data. The paper distinguishes the model initially trained without actions from the V-JEPA 2-AC variant, adapted with interaction data for action-conditioned prediction and robotic planning. The authors present grasping and placing tasks with visual goals. [S9] The distinction matters to the architecture: observing how things evolve and estimating the effect of one's own intervention are related problems that nevertheless require different information.

Returning to the robot with the cup, a learned model can help select a movement and anticipate its visual result. The symbolic world pattern, however, can retain that the meeting has moved, that certain people have confirmed the new room and that an instruction concerns a particular cup. This information comes from conversations, calendars and the rules of the activity. To use it together with the visual model, the system must link the recognized object to the object tracked in the instance and associate the symbolic goal with an observable result.

Such links are where the quality of integration is decided. If the image shows two similar cups, the relevant identity may require an additional clue. If the robot moves the object to the old room, the movement may be flawless while the task is fulfilled incorrectly. An

SWM provides the space in which we retain the relationships of the activity and check the result against them. The learned model contributes the perceptual and predictive capabilities needed to connect that activity to a variable environment.

We can make this cooperation implementable through the perceptual module's contract. The input may be an identified image, its capture time and the camera information needed for measurement. The output contains candidate objects: the image region, the proposed class, the estimated position with its coordinate frame, and scores whose meanings are declared. The identifier of a detection in an image does not automatically become the identity of the cup in the instance. The module may observe two cups without knowing which is intended for the guest; the absence of this information must remain visible in the result.

Linking takes place in a distinct step. The system compares candidates with objects already being tracked, using continuity of position, observed features and known relationships. If a single association satisfies the adopted criteria, it records the link together with the observation supporting it. If two associations remain plausible, it retains both possibilities and may ask for the indication “the one beside the plate.” It does not invent a new object just because the detection number has changed between images. If the association remains unresolved, operations requiring that identity do not yet have the necessary premise. A correct subsequent inference does not retroactively make the perception it started from certain.

A predictor has a different contract. For the robot, the input may include the estimated state of the scene, a sequence of candidate movements and the calculation horizon. The output describes possible future states at specified times, together with the meaning of the uncertainty provided by the model. We retain the model's identification and the assumptions of the calculation so that we can explain why two predictions differ. If required variables are missing or the situation fails the applicability checks defined for the model,

the component returns an unusable or unknown result. Such an output does not authorize the automatic reuse of an old prediction for the new scene.

A delay explained at different scales

A stop display shows that the bus will arrive in seven minutes. Someone on the bus needs to be present at the start of a meeting, while the transport coordination center is discussing a route change because of roadworks. The same journey enters three activities: estimating arrival, organizing the meeting and choosing an intervention in the network. A good prediction for the first question can become a premise for the others if we retain what it estimates and the conditions under which it was obtained.

A predictor learned from route history can use the vehicle's position, the time and recent delays to estimate the remaining time. A dynamic traffic model can represent speeds, traffic lights and queue formation; at a finer scale, vehicle motion involves physical relationships between distances, speeds and accelerations. A symbolic model can retain which stops must be served, which sections are closed and whether an alternative route still allows the goal to be reached. Numerical calculation and explicit relationships meet in the instance of a particular journey.

Suppose a revised estimate shows a delay of twenty minutes. The traveler's assistant can check whether the meeting has an opening part at which their presence is indispensable. The transport system can compare alternative routes. The two decisions use the same prediction but have different actions and criteria. The pattern of each world establishes what can be changed and what result would count as success. A delay message should not automatically become cancellation of the meeting or a route change.

The example shows why families of models can overlap. A physical model can have parameters learned from data. A symbolic model can contain probabilistic relationships. A neural prediction can be

incorporated into an explanation indicating its provenance and assumptions. The words “symbolic,” “numerical” and “learned” describe aspects of constructing and using a representation; they do not divide all possible systems into unrelated compartments.

The difference between prediction and intervention becomes clear in a hypothetical observation about intersections: records show large delays precisely where officers are sent to direct traffic. It would be premature to infer that the officers' presence causes congestion. Sending them may be a response to congestion that already exists. To estimate the intervention's effect, we need assumptions about the dependencies between these events or appropriate experimental observations. Causal models, developed in the tradition presented by Judea Pearl, provide a language for explicitly formulating these assumptions. [S31]

In an SWM, the description of an intervention must specify which mechanism we change. “There are few cars on this street” may be an observation. “We restrict access to this street” introduces an action whose effects can propagate to neighboring streets. A prediction learned from days without such restrictions requires additional justification for this scenario. The dynamic model can represent the redistribution of flows, while the symbolic model retains the restriction, its period of application and the routes' obligations. We can thus identify precisely which component supports each part of the result.

This diversity becomes easier to use when we compare the questions instead of seeking a winner among families of models.

Question about the journey	Representation that contributes directly
When the bus is likely to arrive	A predictive model and an estimate of its uncertainty
How the queue forms at the intersection	A model of traffic dynamics
What would change if we closed a street	A model with specified interventions and causal assumptions
Which routes satisfy the conditions of the trip	Relationships between places, intervals and permitted operations
What must be changed in the meeting plan	The link between arrival, participants' roles and the meeting's purpose

The final explanation can retain these origins. The arrival time comes from a predictor, the street closure from adopted information, and the need to be present at the start from the organization of the meeting. If the user specifies that they can attend the first part by phone, the plan can change without the traffic prediction having improved. We have discovered where freedom of choice exists: in the link between worlds, not necessarily in the phenomenon we are anticipating.

How we preserve meaning when changing representations

Consider again the sentence “Take the cup prepared for the guest to the meeting room.” A language model can identify an action, an object described by its role and a destination described by a relationship. To arrive at an operation, the system must resolve these de-

scriptions in the current context. The same sentence may refer to different objects tomorrow. Interpretation requires access to the instance, and the instance must be able to receive the new information extracted from the message.

The formalizer's contract can require each proposal to contain the operation, arguments, resolved references and the message fragments supporting it. The input includes the message, the selected context and the vocabulary of the pattern used. If “the meeting” refers to two events, the output retains the unresolved reference or offers two candidate interpretations. A plausible text produced by the model is no substitute for the missing identifier. This contract enables local checking of the proposal and the formulation of a precise question before the chosen meaning is used by other components.

The neurosymbolic architectures discussed earlier, such as Neuro-Symbolic Concept Learner, also offer a way to locate errors through the components that produce and interpret representations. [S10] We can apply this idea to our example: if the robot chose the wrong cup, we must distinguish an incorrect perception of the scene from an incorrect resolution of the expression “for the guest.” The image may be recognized correctly while the association with the object's social role is wrong.

If the old room was chosen, we check the context update. If the target is correct but the movement fails, we check control and observations. This analysis guides evaluation: we can retain the same perception and change the verbal description, or retain the same instruction and change the scene. We thus observe where the relevant distinction is lost. An incorrect answer becomes information about the link that must be improved, and learning can be directed toward that part of the system.

A related question concerns how we retrieve a relevant situation from memory. “Elena invites Radu” and “Radu invites Elena” contain the same names and the same verb but imply different relationships. A representation retaining only the presence of the ele-

ments loses precisely the distinction needed to answer who should receive the invitation. In a representation with roles, Elena is associated with the role of host in the first case and guest in the second.

Tony Plate's holographic reduced representations offer a way to encode such structures in fixed-dimensional vectors. Binding operations connect roles to values, while retrieval attempts to reconstruct the relevant elements. [S30] We can imagine a memory that proposes, among many situations, the meeting matching the current message. The symbolic instance would then retain the exact identities and relationships of the adopted candidate. The intended benefit would be fast retrieval of the appropriate context, verifiable through questions that change the roles while retaining the words.

In the proposed architecture, we associate each representation in memory with the identifier of the symbolic situation from which it originates. The request about the meeting in which Elena invites Radu is encoded with those roles, and the search produces a few candidates ranked by match. For each, we open the exact record and check the identities, roles and required temporal context. The situation in which Radu is the host and Elena the guest is rejected even if vector proximity brought it among the first results. Distributed memory narrows the search; explicit verification decides whether a candidate satisfies the relationships in the question.

The similarity score indicates the match in the chosen representation. It is not the probability that this meeting is the one the user intended. If two meetings with the same roles remain, we need a date or another clue, not a conversion of the score difference into certainty. And if no candidate passes verification, we can expand the search or consult the symbolic registry directly. Exhausting a batch of candidates does not prove that the sought situation does not exist: approximate retrieval may have omitted it. At the budget limit, the correct result is “we have not identified the context,” with the attempts retained so the process can continue.

This separation gives us two distinct things to evaluate. The verifier must reject role reversals and unsuitable contexts. Memory must bring the relevant candidate into the examined batch often enough, at an acceptable cost. Exact verification can ensure that acceptance is correct relative to the recorded data, but cannot itself retrieve candidates the memory did not propose. The extension experiment thus has a clear object: how much search we save without losing the situations the conversation needs.

Here we have a concrete criterion for an SWM extension: if memory helps only when names differ but confuses host and guest, it does not solve the proposed task. If it preserves roles and correctly retrieves the situation from among similar alternatives, it can become a useful component. Distributed and explicit representations are thus linked through a precise operation with a result we can measure. The choice of a technique starts from this need, not from the resemblance of its name to a metaphor about the world.

SOP can remain a shared notation even when components work internally with vectors, equations or graphs. A thread can identify the object, a reference can link an observation to that object, and a declaration can specify which operation we request from an interpreter. The meaning of the result, however, must return to the same relationship with the world: the case for which it was calculated, the assumptions under which it was calculated and its role in the decision. A latent coordinate and a statement about an object do not become equivalent merely because we put them in the same application; the link between them is part of the model we build.

This perspective makes cooperation between models easier to design. We can learn where regularities are available in data, calculate with explicit relationships where meaning must be tracked, and connect results through verifiable identities and conditions. When the architecture continuously receives observations from a home, building or piece of equipment and uses them to track that

system's evolution, the discussion naturally reaches digital twins. There we will see how these roles are organized into a persistent relationship with the real world.

9 When the representation tracks the real world

A public meeting begins at five o'clock in a library. The room appears free in the calendar, has enough seats on the building plan, and the management application shows that the heating is working. Yet someone entering the room finds it is still cold. Each application may correctly display the information it holds. The problem lies between them: room availability, the operation of equipment and the conditions in which people can use the space are related but different things.

The question “Will the room be ready in time?” requires a representation in which these links can be tracked. We need to know which room was reserved, which installation heats it, where the displayed temperature comes from and how long the desired change would take. We also need to know whether switching on the heating was merely planned or the device confirmed it. Here we encounter one of the most concrete uses of a world model: the representation is maintained through contact with a system that continues to evolve outside the conversation.

Digital twin technologies build precisely such links between tracked systems and their representations. NIST discusses digital twins through their ability to represent states and transitions of entities and through the conditions under which we can trust their

use. [S11] For our book, the productive question is how world patterns and instances can participate in this work. The library will need information from several worlds, and the symbolic part can make the passage from a measurement to a decision about the event intelligible.

An observation needs a place in the world

A number such as “18” can enter the system without yet answering any useful question. It may be the air temperature in the room, the water temperature in the installation or a value measured yesterday. An interface that displays it attractively does not resolve this ambiguity. Before using it to prepare the room, we must reconstruct the relationship between the number, the measured property, the object it concerns and the time of observation.

Suppose the sensor labeled “T2” has been moved from the corridor to the reading room. Its history now contains observations about two locations. If we update only the sensor's displayed name, we risk reinterpreting the past as though all measurements had been made in the reading room. The instance needs to retain the change in the link between sensor and space. This example shows why the identity of a device and the identity of the thing observed deserve separate representation: the same instrument can participate successively in different situations.

We can organize processing in a service that receives events and proposes instance updates. Each event carries an identifier, the source, measured property, value, unit, observation time and receipt time. The service consults the links valid at the observation time to identify the room. It then checks whether the unit can be converted into the one used by the model, whether the value has the required form and whether the message has already been processed. It retains the original alongside the converted value. An “18” without a unit or identifiable location remains an event to cla-

rify; it does not automatically become the room temperature. This first stage produces either a usable observation with provenance or a precise reason for suspending its use.

An important part of today's infrastructure provides precisely the objects and relationships sufficiently precise to construct these links. OPC UA defines an information model with objects, variables, references and types, as well as associated operations and events. An application can thus receive values in the context of a described equipment structure. [S20] For our library, this structure could make the temperature provided by the installation accessible; the pattern used by the application would specify whether the property represents the room air or the water circulating through the radiator.

In a larger building, hundreds of such relationships arise, often named differently by suppliers. Brick proposes a semantic vocabulary for physical, logical and virtual building assets and the links between them. The description distinguishes relationships such as location, composition and the connection of equipment to its measurement points. [S25] This is a substantial affinity with the idea of a pattern: the reusable meaning of relationships allows the same question to be formulated in differently organized buildings. “What is the relevant temperature for this room?” can be resolved by following relationships rather than relying on manual interpretation of every technical label.

Time adds a second difficulty. At 16:10, a message measured at 16:02 may arrive after the system has already received the observation from 16:08. Retaining the last value to arrive would, in this case, turn the current representation toward the past. Event streams and periodic readings must be linked to an update rule that accounts for observation time. The history receives the delayed measurement; the state used for the current decision is reevaluated according to the situation, not merely message order.

Specifically, the service records the observation from 16:02 in the history and checks which estimates depended on that interval. If the estimator supports recalculation, it resumes calculation from an earlier affected time and advances it to the present, including the observation from 16:08. Only the result of this replay can modify the current estimate. If the procedure cannot reconstruct the interval, it retains the historical correction and flags the estimate for checking. In both cases, the application explicitly receives what has changed: the history, the current estimate or both. Late arrival thus does not secretly acquire the meaning of a recent cooling of the room.

NGSI-LD, standardized by ETSI, handles contextual information through entities, properties and relationships and allows their evolution over time to be represented. It distinguishes information about times, including observation time, and defines queries over this evolution. [S24] For an SWM, this infrastructure offers a way to retain the context from which a value originates. The rule deciding whether a measurement can still be used to prepare the event remains tied to the phenomenon's pace and the particular question.

Once identity and time are established, we can maintain an estimate of the state. A sensor provides a local observation; a model can estimate the temperature relevant to the whole room, using the sensor's position and previous evolution. When data are temporarily missing, the model can continue predicting, and a new observation can correct the estimate. The symbolic part retains the distinction between measurement and estimate and makes the latter's dependence on the model visible. The application can thus say that the temperature is estimated over the relevant interval or that it needs a new reading, depending on what it must decide next.

The adopted estimate has a version and a reference time, together with the observations and assumptions supporting it. Two incompatible readings are not reconciled by arbitrarily choosing the last to arrive: the application checks whether they measure the same thing and whether there is a justified criterion for preferring one. Until then, it can retain the conflict or an interval sufficiently wide

for the admitted data. Updating completes this step when it has produced a usable state or identified the missing information. Only affected evaluations, such as preparing the room for the meeting at five, enter the work agenda.

Descriptions that can be used in several buildings

If every room required the invention of a new language, building the digital twin would soon get bogged down in integration work. A library has different rooms, but many questions recur: which space can be reserved, which equipment serves it, which measurements describe it? For such questions, the reusable description is as important as the current data. It allows the second building to benefit from what was understood in the first.

Digital Twin Definition Language, used by Azure Digital Twins, describes models with properties, relationships and components. Representations of particular entities, linked in a graph, are created from the models. [S21] We can define what information characterizes a room and then represent the ground-floor reading room or the upstairs event room. In the book's terms, we find here part of the relationship between pattern and instance: shared definitions enable consistent interpretation of many particular cases.

The meaning of a room ready for an event, however, also requires relationships between properties. Capacity concerns a particular chair arrangement, the required temperature concerns an interval, and acceptance of the reservation depends on other uses of the space. The world pattern can connect the structural description to these conditions. A user's question thus reaches the properties that matter and the operations to be performed. If the room can accommodate twenty people in its current arrangement, merely changing its declared capacity to thirty does not move the chairs or prepare the space.

In industry, the Asset Administration Shell organizes asset descriptions through identifiers, submodels and semantic references. The specification distinguishes type assets and individual assets, as well as submodel templates and their instances. [S19] This separation is also useful for building equipment. Two pumps may share design characteristics yet have different maintenance histories. The shared description says how certain properties should be interpreted; information about an installed pump says what we know about that particular unit and which interventions it has received.

A symbolic instance of the building is therefore not a copy of each device's product sheet. It connects the information relevant to the activity being tracked. When a pump is replaced, part of the reusable description may remain the same, but the installed equipment's identity and history change. When a room's use changes, the equipment may remain physically the same while the conditions by which we evaluate the space become different. Pattern and instance help us locate the difference without rewriting the entire building for every change.

There are also questions for which geometry becomes essential. If we move the presentation table, we want to see whether space remains for the audience and whether the screen is visible. OpenUSD offers mechanisms for scene description and composition through which reusable objects and contributions can be brought together in variants. [S27] A designer can thus explore several layouts starting from the same building. In an integration with SWM, scene identities can be linked to reservations, equipment and event conditions. The visual proposal and the arrangement actually observed retain their distinct roles.

These parallels show that the idea of reuse does not begin with SWM. Mature mechanisms already exist for describing object types, relationships and instances. Our proposal examines how such descriptions can be used in a process of understanding and reasoning: from “we are moving the meeting to the large room” to

identifying the necessary changes, the conditions still unknown and the explanation the organizer receives. The description of the world thus becomes active in conversation.

A room's future must be calculated

At four o'clock, the room is correctly identified and we have a recent measurement. The temperature is 16 degrees, and the organizer wants at least 21 degrees at five. A question now arises that the equipment inventory and sensor history cannot resolve by themselves: if we switch on the heating now, what will happen over the next hour? The answer requires a model of evolution connected to the conditions in which the room will operate.

To follow the complete calculation, we use a numerical example constructed for this explanation. We assume that, under a fixed heating regime and with the windows closed, the temperature tends toward 24 degrees. Every fifteen minutes, the difference between the current temperature and this value decreases by a quarter. In a real application, the parameters would have to be identified from the room's behavior; here they are chosen so the reader can check each step.

$$\text{next temperature} = \text{current temperature} + (24 - \text{current temperature}) / 4$$

Initially, the difference is eight degrees, and a quarter of it is two: after the first interval we obtain 18 degrees. The next interval adds a quarter of six, taking us to 19.5. Continuing in the same way gives the evolution below. This is a calculation on a hypothetical instance starting from the observed state, in which the assumption about the heating regime remains valid.

Scenario time	Calculated temperature
16:00	16 degrees
16:15	18 degrees
16:30	19.5 degrees
16:45	Approximately 20.63 degrees
17:00	Approximately 21.47 degrees

The model now offers a difference relevant to the decision. Switching on at four o'clock leads, under these conditions, to exceeding the target by the start of the event. If we assume the room remains at 16 degrees until 16:15 and heating begins only then, by five we have only three intervals of evolution and approximately 20.63 degrees. The fifteen-minute delay changes the conclusion. The link between the event schedule and the thermal model has made visible a dependency that the calendar alone did not contain.

To reproduce this calculation, the application prepares an identified scenario: the initial state version, proposed switch-on, closed-window assumption and 17:00 deadline. It selects a model whose inputs, units and conditions of use correspond to the scenario and fixes the parameter version. The interpreter advances the simulated clock, applying the transformation at each interval. It retains intermediate results separately from real observations. In our example, it stops after four steps, at the required deadline. An indispensable missing input suspends calculation; a model unable to continue returns the reason for stopping. The application can thus request the outside temperature or choose another model instead of presenting a number without foundation.

In an application, model selection depends on what we want to anticipate. Physical equations can describe heat exchange, the thermal inertia of walls and the effect of ventilation. A learned model can estimate the room's response from previous episodes, using the outside temperature and the installation's operating regime. A discrete-event simulation can track when groups enter, when the space's use changes and when staff are available for preparation. Each calculation has inputs and results that must be linked to the same times and objects.

Functional Mock-up Interface, FMI, provides a contract for exchanging and executing dynamic models. In Model Exchange, the environment importing the model supplies numerical solving; in Co-Simulation, components can bring their own solving mechanisms, while the environment coordinates data exchange and simulation advancement. [S26] For a building, this enables cooperation between models of rooms, installations and control without all of them being implemented in the same tool. The symbolic part can organize the scenario: which component is simulated, which assumptions have been chosen and which question receives the result.

On return, the result carries the prediction label, the scenario identity, the time for which it is valid, the model used and the relevant assumptions. If the model supplies an uncertainty estimate, this accompanies the value and the method by which it was obtained. The interpreter does not record “21.47” as an observed temperature. The application checks whether relevant inputs have changed in the meantime; a correct calculation for the old situation may need to be repeated. It then compares the applicable result with the event's requirement and retains the conclusion's justification. A subsequent change in time or initial state makes it visible that this conclusion must be recalculated.

A prediction becomes more useful when we know how it behaves when confronted with observations. If the temperature measured at 16:30 is 18.8, the difference from 19.5 can guide a check: did the heating start on time, is the window closed, have outside condi-

tions changed? If, for the five-o'clock scenario, the model analysis supports an interval between 20.5 and 22.4 degrees, the target of 21 remains uncertain. The system has a concrete reason to check the room earlier or prepare an alternative. The practical value of prediction also lies in this ability to show when we still have time to change the plan.

The comparison uses the same property and the same time: the observation from 16:30 is compared with the corresponding simulation step. A deviation beyond the tolerance established for the model triggers a check of inputs and assumptions; by itself, the deviation does not identify a cause. Confirmation of an open window requires a scenario compatible with the new situation. Repeated mismatch, with verified inputs, may require parameter reestimation. In both cases, a new version is created and prediction resumes, retaining the previous calculation for comparison. The process stops when a usable result exists, when a necessary observation is missing or when the computation budget established by the application has been consumed.

A command's story continues outside the model

The organizer says “switch on the heating,” and the assistant confirms that it has sent the request. Twenty minutes later, the room is still cold. From the conversation's perspective, the instruction seems resolved; from the building's perspective, we do not yet know whether the installation received it, accepted it or had its regime subsequently changed by another control. For the assistant to be useful, the representation must also be able to track this part of the story.

Eclipse Ditto explicitly retains current properties and desired properties for device features. The documentation leaves the logic by which the desired state is achieved to the application or device. [S23] This distinction allows the temperature target, the transmitted request and the observation to coexist without being confused.

In our instance, 21 degrees may represent the event's requirement, approximately 21.47 the result of a simulation for five o'clock, and 18.8 a measurement at 16:30. Each value answers a different question.

The numerical interpreter produces the prediction; the executor communicates with the installation. The application tracks responses and observations showing whether the effect occurred. If the device confirms the regime change, we have information about the command. If the temperature starts to rise, we also have information about physical behavior. A communication interruption between these times requires retention of an outcome that is still unknown. This representation allows a state check before repeating the command and a faithful explanation for the person who requested it.

The execution request receives its own identifier, target device, requested regime, deadline and preconditions. Immediately before sending, the executor checks the available state: the installation is the same, has not entered maintenance, and the request remains authorized and relevant. A changed precondition stops sending and returns the decision to the application; an unknown decisive condition requires checking. Confirmation of receipt is associated with the command identifier; confirmation that the regime was applied is recorded separately. An explicit rejection produces a known failure. A timeout without a response produces an unknown state requiring verification before resending. Subsequent observations separately show whether the room is warming and whether the target has been reached. These close the evaluation of the intended effect, rather than merely confirming the message.

Updating can also affect other representations. Azure Digital Twins documentation shows how events modifying one twin can be processed through functions to update related twins in the graph. [S22] In the library's case, a temperature change can update the room-readiness assessment. A change to the event time can require a new simulation. The semantics of these effects belongs to the ap-

plication: the graph supplies the relationships, while rules and code specify which consequences are calculated and which information is retained.

Dialogue can now participate more precisely in operating the building. The message “we are coming an hour earlier” changes the preparation deadline without automatically changing the temperature target. “We are moving the meeting to the small room” changes the relevant space and the installation serving it. “We opened the windows for ventilation” brings an observation that may invalidate the thermal scenario previously used. The neural component identifies the message's probable meaning; patterns and instances allow its consequences to be tracked through calculations, questions and actions.

The same building can thus be tracked from the world of event scheduling, the world of installation operation and the world of energy consumption. The room is shared, but a confirmed reservation, an estimated temperature and a calculated cost have different justifications. Connecting them enables an answer such as: “The room is reserved, but reaching the desired temperature before the new time depends on switching on the heating immediately; the small room is already within the required range.” Beyond a screen displaying values, we have an explanation of the available options. The next problem is how to retain this precision when a decision crosses into still other worlds with different rules and responsibilities.

10 How worlds work together

At nine o'clock, someone in the company asks the assistant, “Are we ready for this afternoon's visit?” The calendar shows a confirmed meeting. A message from production announces a delay. A measurement used in the product presentation has been corrected in the research report. A one-word answer would conceal the situation. To help, the assistant must discover which of these changes influence the visit's purpose and which options remain open.

We return to the electric bicycle company to follow the architecture at work. This time, the customer actually wants to try experimental unit B17 on Friday at fifteen hundred. It appears in scheduling as a resource needed for the meeting, in product preparation as an object within a process, and in research as the subject of an experiment. The shared identity allows information to be connected. Its meaning must be preserved along this connection.

A result must retain its meaning as it circulates

Production says B17 will be ready after twelve if the final check is completed successfully. Scheduling can use this information to evaluate the meeting at fifteen hundred. The received result, however, is a conditional estimate, not an observation that the bicycle is already ready. The link between worlds must preserve the object, use, time and condition. Otherwise, an abbreviated formulation such as “available at noon” may lose precisely the dependency that needs to be tracked on Friday morning.

Research supplies another contribution: the result of a comparison concerning the bicycle's range, obtained in a described experiment. This information can support the presentation the consultant gives the customer. For a claim about the customer's daily route, we need to know to what extent the conditions of that route are covered by the experiment or a validated model. The assistant can separate

what can be reported now from the question requiring additional information without reducing the entire discussion to a verdict about the product.

In turn, the conversation with the customer can change how we use these results. We learn that the main interest is the riding position and uphill behavior; the experimental battery is an additional curiosity. Another bicycle, already prepared, may satisfy the main purpose. This information opens an alternative for the meeting, although it changes neither B17's status nor the experiment's result. Clarification has its effect in the relationship between purpose and options.

To organize such transitions, we can describe a contract between worlds. The word “contract” here names a precise understanding about the meaning of the transmitted result: what it concerns, when it is valid, what it depends on and what the recipient can do with it. Some conditions can be checked mechanically, such as the unit's identity or the compatibility of times. Others depend on a domain rule, such as using a technical check for a particular type of demonstration. The contract becomes part of the inspectable link between instances.

In a large organization, data may come from systems identifying the same thing differently. The calendar uses a commercial name, production a serial number, and research an experiment code. Associating them requires a justified relationship. There may be several units with the same commercial name and several experiments on the same unit. Correct mapping makes it possible to ask “which result concerns exactly the bicycle we will use?” and prevents name similarity from becoming an assumed identity.

This architecture can use a graph for links. Ontologies and rules can express reusable meanings, while validation can check the requirements imposed on the representation. OWL and SHACL provide established tools for these roles, with distinct questions about consequences and satisfaction of data conditions. [S5, S6] In

SWM, they can support parts of the pattern and instance verification; coordination then extends to time, assumptions and the operations through which the situation is changed.

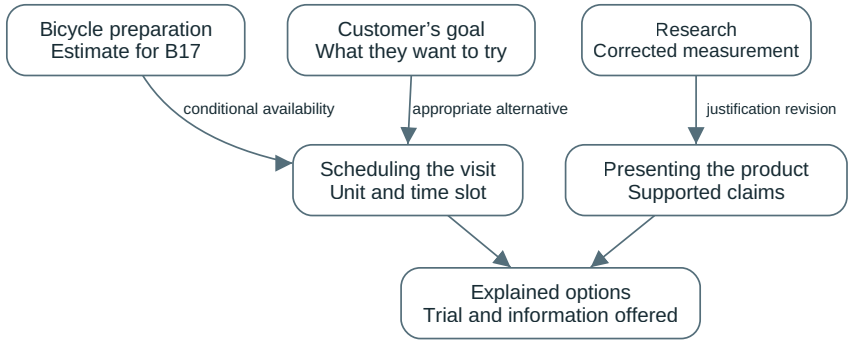


Figure 3. The preparation delay and correction of the measurement affect different conclusions. The customer's purpose can open an alternative for the visit.

A change crosses only the relevant links

On Friday morning, we learn that B17's inspection is extending until sixteen hundred. The conclusion that the unit will be available at fifteen hundred must be reopened. The meeting with the customer, however, remains a separate commitment with several possible continuations. We can propose another unit, a presentation without a test ride or a later time slot. Correct propagation of the change reaches the affected decision and updates its premises; the choice of continuation depends on the customer's purpose and the organization's responsibilities.

This mechanism can be followed as a network of justifications. “The visit can use B17” rested on a readiness estimate and a meeting interval. One premise has changed, so that conclusion is withdrawn or reevaluated. “The customer wants a test ride” has another source and remains valid. “The consultant is available” has its

own dependencies and may remain valid. The system need not restart everything it knows about the customer from scratch, but must identify where the current plan's justification broke.

That same morning, research corrects a measurement. The range comparison is recalculated, and material using the old value must be revised. This change follows a different path. It may influence what we say about the product without influencing the bicycle's mechanical availability. If the inspection instead discovered an operating problem, the link to the demonstration would be direct. To choose the appropriate response, the model must represent the nature of the finding, not merely receive a shared indicator called “problem.”

Some changes remain local. Research may change the form of a graph while retaining the same results. The consultant may receive a new contact number for the customer without a change of visit location. An internal preparation step may be replaced by another equivalent for the demonstration conditions. Sending all these changes to all components would create noise and conceal important events. Declared links enable selective propagation, explainable through the meaning of the dependencies.

Some requests also cross boundaries of authority. “Stop the experiment earlier so we have the bicycle for the customer” is a proposed change to the research activity. The calendar can show the benefit, but the decision must account for the observations that would be lost and the person responsible. The assistant can construct a scenario in which the experiment is interrupted and compare its consequences with those of using another bicycle. The scenario becomes an adopted change after the relevant decision, and its execution must be tracked separately.

The same principle is useful in a personal assistant. Moving a trip can affect meetings, people waiting for us and preparation of the home. The system can identify all these dependencies, but each action has its own meaning and conditions. Updating the likely arrival time differs from canceling a meeting on behalf of all parti-

cipants. An architecture retaining the difference can be active and useful without turning every logical consequence into an automatic action.

How the application coordinates a complete cycle

At nine o'clock, two changes arrive: the delay in B17's preparation and the correction of the research measurement. How does the system process them without confusing the results or losing the already confirmed visit? We can now bring the book's mechanisms together in an integration algorithm. It starts from identified events, updates the appropriate instances, executes affected operations and produces an explanation together with the actions still needed.

When the application starts, each pattern is loaded and checked together with the operations it requires. The application maintains a registry of interpreters: for each operation, which inputs it accepts, which results it can produce and under which conditions it can be used. An interval check, a range estimate and a calendar command have different contracts. If a circuit requires an unknown operation, the system reports it as unsupported; it cannot consider the condition satisfied because it omitted the calculation. We thus learn when connecting the components whether the description has an effective means of being processed.

When the message about B17 arrives, the adapter identifies the source and unit, and formalization proposes changing the readiness estimate. The production instance receives the change after its meaning and references have been checked. The reverse dependency link leads to the result transmitted to scheduling: conditional availability at fifteen hundred. The application marks this conclusion as requiring reevaluation and places the operation on the scheduling instance's agenda. It does not send the message to every world or cancel the visit yet.

For each operation, the coordinator constructs a request identifying the instance, pattern version, inputs and question. If alternatives are being explored, the request indicates the scenario. The interpreter receives a stable view of the necessary data and returns the result together with the dependencies used. These dependencies must also include the collections queried: the conclusion that a slot is free may depend on the absence of other reservations, not just on the records found. A new reservation in that slot can invalidate the result even if no previously read object has changed.

In parallel, the research correction activates recalculation of the comparison and revision of the sales material. The two branches can advance independently as long as they do not use each other's results. Within each, operations requiring values not yet produced wait. The coordinator tracks this order through computational dependencies without deriving it from the order of threads in the circuit. For groups of recursive rules, it uses the method declared by the interpreter, such as applying monotone rules until no new facts appear within a finite domain. Other forms of recursion require the evaluation method and stopping conditions declared by the interpreter. A cycle between external commands does not receive this treatment: repeating a command has effects and requires an explicit protocol.

The interpreter's result is checked before adoption. If the visit has moved in the meantime, an evaluation made for fifteen hundred may remain useful in the scenario history but does not automatically describe the current schedule. The application compares the relevant versions or dependencies, then accepts the result or repeats the operation. Verification and adoption are protected together in an atomic local operation: relevant data cannot change between the two stages without adoption being refused. An unrelated change, such as the customer's telephone number, does not require repeating the availability calculation. This rule enables parallel processing while retaining a verifiable basis for every conclusion.

A result is not reduced to a value. It may contain a calculated conclusion, missing data, a conflict between premises, a rejection of the model's applicability, an error or termination at the computation limit. In our example, “The updated estimate no longer supports B17's availability at fifteen hundred” is the appropriate conclusion for the new information. “Range cannot yet be recalculated” may result from the absence of a replacement measurement. These results require different continuations. The first activates a search for an alternative for the test ride; the second keeps the sales claim pending the data.

The agenda is cleared only of operations that have been resolved or moved to an explicit waiting state. For each suspended operation, we retain what would allow it to resume: a measurement appearing, the user's answer or a result from another system. Thus, “we have no more executable calculations now” does not become “we have resolved all the questions.” The answer to the person can specify the established part and the condition still open. This is a useful stopping criterion in a conversation in which the continuation may come several hours later.

Both the adopted state and the log of relevant events and ongoing operations remain in application memory. The local change and the recording of tasks arising from it must be saved together or through a mechanism allowing the agenda to be safely reconstructed. Otherwise, an interruption between saving the new time and scheduling recalculation would leave an apparently updated instance with old conclusions. Events and tasks have stable identities so that resumption recognizes what has already been applied and does not generate a new command for the same result. After restart, the system recovers unfinished tasks and checks unresolved external outcomes. Persistence also concerns continuity of processing, not merely retention of the conversation text.

How a decision reaches a verified effect

The customer accepts the alternative bicycle. This reply resolves a choice, but the system must establish which operations acceptance produces in the existing applications. The calendar may require changing the resource associated with the visit; production may require reserving the unit for the test ride. The coordinator constructs the corresponding commands and links them to the same decision, retaining each result. If only the first succeeds, the plan is partially realized and an obligation remains to be resolved.

Before transmission, the executor checks the preconditions that can be controlled in the relevant service. To avoid concurrent allocation, local verification must be complemented by the operation of the service owning the resource: for example, accepting the reservation only if the unit is still available. A favorable answer obtained a minute earlier may be insufficient. The request includes an operation identity, and the response is correlated with it. If the service supports checking a request's state or handling repetitions without a new effect, the integration uses that capability.

A connection interruption after transmission creates a precise situation: we know what we requested, but do not yet know whether the operation was performed. The executor retains the request in this state and seeks confirmation before deciding to repeat it. An explicit rejection follows another branch and may activate an alternative. Through these states, the application can explain progress without confusing a communication error with a rejected reservation or a confirmed one.

Let us follow the favorable continuation. The calendar retains the time of fifteen hundred and confirms the alternative unit; production confirms its reservation for the test-ride interval and that it has passed the inspection required for the demonstration. The visit can proceed. However, the sales presentation still contains the value withdrawn by research. The two confirmations do not make

that claim true. The system locates the remaining work: the battery material must be revised, and the unsupported claim cannot be presented as a current research result.

The final explanation is constructed from the user's question and the justifications of the relevant results. It can say that the meeting is retained, that B17 was replaced to keep the time and that the battery data await correction. For each claim, we can recover the source, rule or confirmation supporting it. The language model can formulate the answer naturally, but its content must be checked against these results: “ready” cannot be introduced as a summary of an estimate that remains open.

This architecture can begin in a single process, with an instance store, a persistent agenda, local interpreters and a few adapters to services. Distributing the components is a later choice, dictated by volume, response time and the organization of systems. Responsibilities remain the same. We can add a new simulator or another search mechanism if it declares the operations it understands and returns results with the necessary meaning and dependencies. Extensibility comes from this contract, not from assuming that any algorithm can interpret any circuit.

The strength of integration can be tested through a very concrete scenario: we delay a result, change one premise in the meantime and restart the application before a command is confirmed. The system must reject use of the now-stale result, repeat the affected calculation and recover the command's state. If it does these things, we have built an essential part of the continuity promised by SWM. It remains to understand how we acquire and improve the patterns that give this mechanism meaning.

11 How we learn better patterns

Two teams seemingly repeat the same experiment and obtain different results. The first reaction is to look for who made a mistake. But the expression “the same experiment” may conceal a difference: another instrument, another software version, different material preparation or a time interval that had not been considered important. To choose the next check, we must be able to say what was shared and what varied between attempts. The mismatch of results thus becomes a question about our representation.

A world pattern should not be viewed as an object completed before the first conversation. It can be designed by people, partially extracted from documents, proposed by a learned model and improved through confrontation with instances. The research problem is to organize this improvement so that a new observation helps us choose between an incorrect value, an omitted condition and an unsuitable rule. Each requires a different intervention, and a useful system should be able to show what exactly it proposes to change.

To see what the system should learn, let us first consider what we retain from a situation and how we recognize that a description has left something important out.

Memory of a situation retains more than words

After a few chapters of a novel, we may remember that a character left before the letter arrived without remembering the exact sentence that said so. Later, a scene may surprise us because the character seems to know the message. We compare what we read with a situation we have constructed mentally. Our understanding has an organization extending beyond the form of sentences: participants, places, time, relationships and access to information.

Research on situation models in language comprehension studies precisely such representations. Zwaan and Radvansky distinguish the representation of the text from the representation of the situation described and discuss how the latter participates in understanding and memory. They also separate knowledge about typical situations from the particular case represented during reading. [S14] The distinction provides a reference point close to pattern and instance. For our architecture, it suggests retaining both messages and the situation constructed from them so each can help verify the other.

Returning to the novel, we can design a mechanism tracking the events through which characters learn certain things. Reading, a conversation or a deduction can add information to a character's perspective. If a subsequent action seems to require knowledge for which we find no support, the system can flag the missing link. The author can decide that a scene is missing, that the character deduced something or that the system's observation starts from an incorrect interpretation. The representation makes the problem localizable and leaves the literary decision in its proper place.

There is also knowledge preparing us to understand a situation before we have all the details. When someone says “I reserved a table,” we expect information about location, time and participants. Marvin Minsky's proposed frames organized knowledge about typical situations together with slots to fill and expectations. [S2] In an SWM, a pattern can have a similar role: recognizing an activity activates relevant questions and relationships through which answers can be used.

An expectation becomes valuable when the system can also recognize the exception. Perhaps the “table” is at an event with assigned seats, and one person's confirmation influences the seating of others. Perhaps the reservation does not guarantee access before a certain time. The particular situation can enrich or correct the initial description. For an assistant, this flexibility translates into the distinction between a value adopted from the message, a convention used provisionally and a condition still to be clarified.

Johnson-Laird's theory of mental models centers on the possibilities compatible with premises and the role of counterexamples in reasoning. [S13] It inspires another architectural choice: before presenting a conclusion as necessary, we can look for an admitted situation in which it does not hold. If the table reservation allows several times and only some fit the participants' arrival, the existence of a good option does not justify every schedule. A question about the exact time can thus have an explainable basis.

These links with psychology are productive when they lead to behaviors we can investigate. We can evaluate whether the system retains characters' perspectives, correctly returns to an episode or finds the alternative that changes a conclusion. We need not assume an identity between software organization and brain mechanisms to use these questions. The benefit is a design direction in which memory, reasoning and dialogue are analyzed together.

A regularity becomes a pattern through what it explains

An assistant observes that a team's last ten meetings each lasted 45 minutes. It may propose this duration for a new meeting. Turning it into a rule excluding every other duration, however, would be a much stronger change. In the first situation, we have an expectation based on examples. In the second, we have modified what the system considers possible. Learning a pattern requires distinguishing these roles before adopting the observed regularity.

The user can explain why the meetings had that duration: the room was available for 45 minutes between two classes, and the team used the entire interval. The regularity now receives an explanation separating the limit imposed by the room from the team's choice to occupy all the available time. If we change the room or day, the limit may change. A description retaining the dependency

on availability generalizes differently from the rule “meetings last 45 minutes.” Both fit the initial examples, but only one says something useful about situations in which conditions differ.

Learning can advance precisely through such contrasting situations. We retain the same participants and change the room. We retain the room and change the interval. If the proposed rule continues to produce the appropriate differences, we have better reasons to use it. If it fails, the missing relationship must be identified. A neural model can propose candidates from messages and examples, while symbolic interpretation allows them to be confronted with cases in which they would lead to different consequences.

We can design this process starting from a learning dossier: messages, documents, instances and questions for which the current pattern gave unsuitable or incomplete answers. The neural model identifies repeated relationships and proposes a precise change. In the meeting example, it may replace the fixed duration with a condition linking the meeting's start and end to the room's permitted interval. The candidate retains the passages and cases that motivated it, the conditions under which it applies and the result it changes. A new statement about Tuesday's meeting, by contrast, remains an update to that instance if we have no basis for a reusable rule.

Before comparison, we check whether the proposal can be interpreted: objects have defined roles, intervals use compatible conventions, and the condition refers to that meeting's room. We then apply each candidate to working copies of the same instances. For each case, we retain the consequences, conditions still unknown and the difference from the verified answer. If we do not know the room's interval, the availability-based candidate may remain unevaluable; this result is not a demonstrated failure of the rule. The comparison must show how many cases it explains and in how many it lacks the necessary inputs.

We also check the rules the proposal retains. If we add the requirement to fit within availability but leave the condition of an invariable 45-minute duration active, the description continues to exclude longer meetings. The candidate must specify what it replaces, not just what it adds. The comparison report thus shows whether the actual change corresponds to the reason it was proposed.

The initial ten meetings may leave both proposals standing. We then seek a case that separates them: the same team uses fifty minutes of a sixty-minute slot. The descriptive hypothesis of an invariable 45-minute duration is contradicted by the observation; the condition of fitting within the interval remains compatible with it. Such an observation does not alone prove that the second hypothesis captures all conditions. If the 45-minute duration were a mandatory policy, the same case would describe a policy violation rather than the refutation of a regularity. We also retain cases not used in formulating the candidate, and when they do not distinguish the proposals we can ask for clarification of the team's convention. The learning result may remain a family of candidates accompanied by the check that would enable a choice.

Sometimes the new world resembles one already known. Preparing a meal and building an application have processes with dependencies: one step may need another's result. The useful similarity concerns this relationship, not the material objects. Dedre Gentner's theory of analogy emphasizes correspondences between relationships and their organization into a structure rather than mere similarity between attributes. [S17] For SWM, this suggests reuse that can explicitly state what it preserves across domains.

We can reuse the idea of precedence for a recipe and a software process. Yet a stage requiring a temperature to be maintained for twenty minutes is not equivalent to a calculation whose duration can be neglected at the time scale being analyzed. If we reduce both to “step B follows step A,” we lose part of the first world's dynamics. A good analogy transfers the relationship that helps and

leaves the description of the difference open. It is precisely this precision that enables extension without turning domains into artificial copies of one another.

In research programs on machines that learn and think closer to human capabilities, model building, composition and knowledge reuse play an important role. Lake and colleagues provide a synthesis of such directions. [S18] SWM can be viewed as a meeting place for these concerns: learned proposals acquire a representation in which we can track consequences and see what exactly has transferred into a new world.

An improved pattern must remain linked to the reason for its change. If we added a room's cleaning duration because one meeting could not start immediately after another, that distinction must be retained in the verification examples. Old instances can be reinterpreted if we want to analyze them under the new rule, but explanations of decisions made then must be able to indicate the description used at that time. Learning thus becomes the accumulation of useful distinctions, with a history we can examine.

Adoption can be organized as a change to the pattern version. We record the starting version, added or modified definitions, comparison results and the domain for which we accept the change. The acceptance criterion depends on the rule's nature: a responsible person may establish a scheduling convention, whereas an empirical relationship requires confrontation with observations. Good performance on examples does not authorize changing an organizational policy. The verified candidate, adopted pattern and a hypothesis still under study remain objects with distinct roles.

When an instance moves to the new version, we identify the additional information it requires and the justifications that used the changed definitions. We supplement the data from existing sources, explicitly leave unsupported values unknown and reevaluate affected consequences. If the new description separates meeting duration from cleaning time, the old total interval does not automatically tell us how long each took. The system can produce a

question to fill the gap, retaining the previous calculation linked to its version. Instances continuing to use the old pattern are not modified merely because the new version exists.

Before adoption, we can also compare the change's effects on the same cases: which answers become different, why, and which new information supports them. This comparison is the useful result for the person maintaining the pattern. They can accept the extension, restrict its domain or retain it as a hypothesis. The original observations remain available in all variants so that a new rule does not manufacture its own support by rewriting the cases that generated it.

Returning to the two experiments at the beginning, suppose we discover different versions of the instrument's software. The old pattern did not retain this information, so the trials appeared identical in the representation. We extend it to link each measurement to the configuration used and supplement the instances wherever records exist. The version difference is, for now, a possible explanation. To investigate it, we can repeat the measurement under comparable conditions, varying the version and tracking the result. We have obtained three distinct things: a more informative description, a hypothesis and a test that can separate explanations. This link between representation and investigation gives pattern improvement a concrete meaning.

Research begins when we can put the idea to the test

A system may produce a convincing explanation for three examples and fail as soon as message order changes. To discover what we have built, we need situations that exercise the mechanism, not just its vocabulary. If we model scheduling, evaluation must include a time correction, two people with the same name, an unconfirmed intention and a change affecting only one meeting. Each case checks a relationship the system claims to retain.

A complete evaluation follows the information's path. First, we check whether the message was linked to the appropriate objects and roles. Then we check whether the representation contains the distinctions needed for the question. We track consequences, the response to corrections and the choice of clarifications. If execution is involved, we check the link between the requested command and the observed result. A correct final answer can conceal an incorrect intermediate step that was accidentally compensated for; retaining the path enables us to identify this fragility.

To compare architectures, we can use the same neural component in different configurations. One receives the conversation history and tools. Another also receives persistent instances with explicit rules. The available resources and information must be chosen so the measured difference can be attributed to the mechanism investigated. We can track answer correctness after revisions, the number of questions needed, operation time and cost, and the interventions required from a person to repair the representation.

Ablation poses a simple question: what changes if we remove a component? Without justifications, does the system still withdraw conclusions affected by a correction? Without separating scenarios, can it distinguish what was merely discussed from what was adopted? Without links between worlds, can it notice that a research change influences the claim presented to the customer? A well-chosen test makes the difference observable. If the task does not use the removed mechanism, the result says little about its usefulness in other situations.

Generalization requires similar attention. Replacing names in a set of examples tests recognition under surface variation. Introducing a new combination of relationships tests something else. Transferring a dependency mechanism from a publication to an experiment can test structural reuse if the meaning of the links is specified. A claim of generality becomes informative when we can name what was preserved, what was learned anew and what work remained for the designer.

The research contribution can be very concrete: an assistant that revises precisely the affected conclusions, retains commitments better, asks more useful questions or makes visible a conflict that another organization loses. Such a contribution does not require inventing all the techniques used. It can result from an architecture that correctly connects established mechanisms and makes them reusable in several worlds. The next step is to choose the worlds in which this organization is worth the effort and can produce a difference people feel in their work.

12 Worlds where thinking this way is worthwhile

At the end of a useful conversation, the user should be able to leave and return without reconstructing everything from the beginning. “Let's continue yesterday's option, but now there are four of us.” In a trip, the participants change and perhaps the transport. In a project, the team changes and perhaps the distribution of work. In a story, a scene changes and perhaps what the characters learn. The same words open different transformations that a system can track only if it has retained the situation and the meaning of operations on it.

From here, we can look again at the book's entire journey. The pattern provides the relationships through which a situation becomes intelligible and explorable. The instance retains the case we are working on. Neural models can help construct them from language and observations, while interpreters can transform them into cal-

culations, checks and questions. Choosing a good application begins by identifying relationships important enough to warrant this explicit organization.

Everyday life has a structure that conversation conceals

A family holiday plan may seem like a collection of preferences until a feasible option must be chosen. Dates depend on calendars, the route depends on connections, accommodation depends on participants, and a change may affect people absent from the current conversation. An SWM can retain these dependencies and help separate an attractive option from one satisfying commitments already made. The benefit is easy to recognize: a correction does not force us to revisit every choice or leave an important consequence hidden.

Personal scheduling has a different concern. Sometimes the problem is not finding a free slot but the meaning of “free.” An hour without an event in the calendar may be needed for travel, preparation or rest. The pattern can represent these relationships when the user considers them relevant. The instance retains the day's particular conditions. An assistant can explain why two meetings that do not overlap nevertheless do not fit, then ask for precisely the information that would make an alternative possible.

Preparing an event connects other worlds: guests, spaces, equipment, content and access. “Everything is reserved” may be true while the schedule remains unfeasible because of setup order or preparation time. Symbolic representation helps when the effect of a choice must be tracked beyond the object on which we made it. In a small team, this work is often held in one person's mind. A system that makes it explicit can support cooperation and continuation of the activity when that person is absent.

The home adds observations and continuous processes. Comfort, consumption and available time can be linked through physical models and declared preferences. A temperature prediction can be related to people's arrival, while discrepancies from measurements can open an investigation. The value of the symbolic part appears when the user receives an explanation such as “the arrival time changed, so I recalculated the switch-on time,” supported by the system's actual states and results.

In all these situations, the organization of worlds must be able to follow how the person lives. A discussion of a trip can return after a message about the home. The purpose, scenario and relevant objects must be recovered, and information changed in the meantime must be taken into account. The interface can remain a single conversation even though several instances and their links are maintained behind it. The simplicity of dialogue is supported by the precision of internal organization.

Creation and research require worlds that can be challenged

An author may want to know what changes if a revelation is moved two chapters earlier. The question concerns characters' knowledge, motives for actions and event order. An SWM can track these relationships and show the scenes in which the new information should have an effect. The author retains the freedom to introduce hesitation, a lie or an unexpected choice. The representation helps them see the structural consequences of their decision and consciously choose what to preserve.

A fictional world makes the relationship between pattern and instance particularly clear. A story may allow an extraordinary rule, such as transferring memories between characters. Once introduced, the rule creates consequences: who can use the ability, under which conditions and what they learn through it. Scenes describe successive situations in the story's instance in which those

conditions can be investigated. For alternative versions of events, we can explore distinct instances. The interest of verification comes from the internal coherence chosen by the author, while literary quality continues to depend on the effect on the reader, pacing and expressive choices.

In document analysis, what we track is a relationship between claims, sources and conclusions. Two reports may use the same figure for different periods. A summary may remove a condition that was decisive in the original text. An SWM can retain the link between the conclusion and the passages from which it originates, then show which reading or association made the result possible. When a new document appears, the system can identify claims to revise and questions that remain open.

Research additionally brings the choice of the next observation. If two explanations are compatible with current results, the useful experiment is the one that can separate them under feasible conditions. A pattern can describe the assumed variables and relationships, while instances retain the trials conducted. Simulation, statistics and domain knowledge contribute their own methods. The shared representation allows us to link a question to the information that would change the answer and retain the reason we chose that investigation.

These applications show that a symbolic model can also support activities in which the world is deliberately open to revision. In creation, change may be an author's intention. In research, it may be required by evidence. In document analysis, it may come from a more faithful interpretation. The pattern must allow its own distinctions to be examined and alternatives to be formulated, while instances must retain the conditions under which each conclusion was obtained.

Organizations and autonomous systems need continuity between decision and effect

A company may have competent applications for scheduling, production and research, with the main difficulty arising between them. A technical result becomes an argument in a sales conversation; a promise becomes a requirement for a process; a delay changes a meeting. SWM is suited to investigating these transitions because it can retain the shared object and the different meanings of the information. A conversation can cross several worlds without reducing them to the same list of states.

In the world of programs, actions can have semantics precise enough for analysis. We can study which properties a transformation preserves, which branch loses a reservation or which sequence reaches an unwanted state. Instances can represent states and paths, while the pattern describes the operations and their conditions. A counterexample can become a concrete test, and a proved property can guide implementation. The link to the executed system remains part of the architecture: the same operation described must be the one performed by the code.

In robotics and digital twins, the representation receives observations, sometimes interpreted by neural models, and predictions produced by a simulator or learned model. Symbolic relationships can organize goals, identities, conditions and explanations. A robot can recognize a person or object, anticipate a movement and check whether an action corresponds to the request. A building can connect room occupancy to the operation of installations. Component cooperation is concrete when their results concern the same objects and times.

The following table links families of worlds to the operation that makes them useful in a system. The user's question indicates what must be represented, while the mechanism indicates where we can begin construction.

World being tracked	Useful operation and question being pursued
Travel and mobility	Propagating temporal conditions and searching for options that remain feasible
Scheduling and commitments	Recalculating the dependencies of rescheduling and identifying missing agreements
Events and cooperation	Planning activities and resources to identify the preparation bottleneck
Homes and buildings	Updating state, simulating and reconciling commands with observations
Customer relations	Matching the request to the resources and evidence needed for confirmation
Production and services	Planning with preconditions and effects, followed by execution tracking
Research and experiments	Maintaining hypotheses and choosing the experiment that separates them
Documents and investigations	Tracking justifications and revising conclusions after a source correction
Narratives and games	Temporal propagation of events and characters' access to information
Programs and protocols	Symbolic execution or abstract interpretation to search for violations

World being tracked	Useful operation and question being pursued
Robotics and autonomous systems	Associating perception with identities, planning and checking effects

The first project can be small without being trivial. For a reading assistant, we choose the question “which conclusions change after this correction?” We identify the necessary thread types: document, passage, claim and justification. We specify how a claim connects to its source and how a justification uses other claims. We construct a few instances from documents a person can verify and declare what correcting or replacing a source means. We already have an operation with observable inputs and outputs, even before formalization is fully automated.

We then connect the parser or formalizer, validator, store and interpreter that tracks justifications. The first complete path must be able to adopt information, calculate a conclusion, receive a correction and update only the affected consequences. We add a case with two independent sources and one in which two documents copy the same source. If the system treats these cases differently for justified reasons, the mechanism begins to be useful. The neural component is evaluated separately for the faithfulness of transforming text into threads so that an interpretation error is not hidden beneath correct propagation.

A second world can be the preparation of a publication. It uses the analysis result but adds versions, approvals and publication operations. We define which conclusion can pass from document analysis into the material being prepared and when it must be withdrawn. We thus check whether the same agenda, justification mechanism and instance infrastructure can serve a new activity while preserving its own rules. Extension now has a concrete object: we reuse the mechanisms, add the necessary meanings and test the link between them.

The book began with a departure moved to Friday. On returning to that conversation, we would want the assistant to be able to say which option we were exploring, why it seemed good, what changed and what still needs to be decided. For such an answer, pattern and instance work together with learning, calculation and observation. Understanding becomes something we can continue, investigate and correct over time. This is the practical promise of symbolic world models and the direction in which they are worth building.

Sources and recommended reading

The references below support the historical concepts and technology descriptions. World pattern is the unifying term chosen in this book. The examples are constructed to explain relationships and operations, not to report experimental results. The parallels between technologies and SWM are architectural interpretations formulated in the text. The technical documentation was consulted on 11 November 2026.

[S1] ml-circuits-research. SOP Lang and the soplang-words-lib project. Reference point for notation with threads, identifiers, properties and references. The semantics of the book's instructional rule vocabulary is specified in chapter 3. Project.

[S2] Minsky, M. A Framework for Representing Knowledge. MIT AI Memo 306, 1974. Frames, typical situations and expectations. Text and MIT record.

[S3] Fikes, R. E. and Nilsson, N. J. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. 1971. Actions and symbolic planning. IJCAI paper.

[S4] Doyle, J. A Truth Maintenance System. Artificial Intelligence, 12, 231–272, 1979. Justifications and revision of conclusions. MIT record.

[S5] W3C. OWL 2 Web Ontology Language Primer, second edition, 2012. Classes, properties and open-world semantics. Recommendation.

[S6] W3C. Shapes Constraint Language, SHACL, 2017. Validating graphs against declared conditions. Recommendation.

[S7] Ha, D. and Schmidhuber, J. World Models, 2018. Learned representations and dynamics models for control. The authors' article.

[S8] Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T. Mastering diverse control tasks through world models. Nature, 640, 647–653, 2025. DreamerV3 and learning from imagined trajectories. Article.

[S9] Assran, M. and colleagues. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning, 2025. Prediction in representation space and planning. Paper.

[S10] Mao, J., Gan, C., Kohli, P., Tenenbaum, J. B. and Wu, J. The Neuro-Symbolic Concept Learner: Interpreting Scenes, Words, and Sentences From Natural Supervision. ICLR, 2019. The authors' page.

[S11] Voas, J., Mell, P., Laplante, P. and Piroumian, V. Security and Trust Considerations for Digital Twin Technology. NIST IR 8356, 2025. Report.

[S12] King, J. C. Symbolic Execution and Program Testing. Communications of the ACM, 1976. IBM Research record.

[S13] Johnson-Laird, P. N. Mental models and human reasoning. PNAS, 107, 18243–18250, 2010. Text.

[S14] Zwaan, R. A. and Radvansky, G. A. Situation Models in Language Comprehension and Memory. Psychological Bulletin, 123, 162–185, 1998. Text on the laboratory's website.

[S15] Clark, H. H. and Brennan, S. E. Grounding in Communication. In Perspectives on Socially Shared Cognition, 1991. The author's bibliography and access to the text.

[S16] Nelson, T. O. and Narens, L. Metamemory: A Theoretical Framework and New Findings. Psychology of Learning and Motivation, 26, 125–173, 1990. Publisher's record.

[S17] Gentner, D. Structure-Mapping: A Theoretical Framework for Analogy. Cognitive Science, 7, 155–170, 1983. Text on the author's website.

[S18] Lake, B. M., Ullman, T. D., Tenenbaum, J. B. and Gershman, S. J. Building machines that learn and think like people. Behavioral and Brain Sciences, 40, e253, 2017. MIT page. The authors' preprint.

[S19] Industrial Digital Twin Association. Specification of the Asset Administration Shell, Part 1. Types and Instances; Submodel Instances and Templates. Specification.

[S20] OPC Foundation. OPC Unified Architecture, Part 3: Address Space Model. Objects, types and relationships of the information model. Specification.

[S21] Microsoft. Learn about twin models and how to define them in Azure Digital Twins. DTDL models, properties, relationships and components. Documentation.

[S22] Microsoft. Set up twin-to-twin event handling. Propagating updates through functions and events. Documentation.

[S23] Eclipse Ditto. What is Eclipse Ditto and Features. Current state, desired state and the application's role in reconciliation. Overview. Properties and states.

[S24] ETSI. NGSI-LD, Context Information Management Framework. Entities, relationships and temporal representations. Specification.

[S25] Brick Ontology. Semantic descriptions of buildings, relationships, inference and validation. Introduction. Relationships. Inference. Validation.

[S26] Modelica Association. Functional Mock-up Interface Specification, FMI 3.0.2. Interfaces for exchanging and executing dynamic models. Specification.

[S27] OpenUSD. Introduction to USD. Composition of scene descriptions, reuse and overrides. Documentation.

[S28] Evans, E. Context Mapping an AI-Based Component. Domain Language, 6 January 2026. Separating domain models and integrating an AI component through explicit translations. The author's article.

[S29] Cousot, P. and Cousot, R. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. POPL, 238–252, 1977. Text and presentation on the author's website.

[S30] Plate, T. Holographic Reduced Representations: Convolution Algebra for Compositional Distributed Representations. IJCAI, 30–34, 1991. Paper.

[S31] Pearl, J. Causal inference in statistics: An overview. Statistics Surveys, 3, 96–146, 2009. Association, intervention and causal assumptions. Text on UCLA's website.